

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE



PROBABILITY AND STATISTICS
(MT2013)

Assignment

**LAPTOP PRICE PREDICTION
USING MULTIPLE LINEAR REGRESSION**

Supervisor
Dr. Phan Thi Huong

Group 05 - CC03
Tran Gia Han - 2052464
Nguyen Ngoc Khoi - 2252378
Dinh Gia Kiet - 2252399
Le Thi Phuong Thao - 2252757
Bui Thai An - 2252001

HO CHI MINH CITY, APRIL 2024



Contents

1	Data Introduction	1
1.1	Data Overview	1
1.2	Features Overview	1
2	Background	2
2.1	Regression models	2
2.1.1	Multiple Linear Regression (MLR)	2
2.1.2	Lasso Regression	3
2.1.3	Ridge Regression	4
2.1.4	Elastic Net	4
2.2	Reference models	5
2.2.1	Decision tree	5
2.2.2	Gradient Boosting	6
2.2.3	Support Vector Machine	6
3	Data pre-processing	6
3.1	Data reading	6
3.1.1	Libraries	6
3.1.2	Importing data	7
3.2	Data analysis	7
3.2.1	Check missing value	7
3.2.2	Data statistic	8
3.3	Correlation coefficients between variables	8
4	Descriptive statistics	9
4.1	Data summary	9
4.2	Data plot	11
5	Inferential statistics	16
5.1	Multi-linear Regression	16
5.2	Weighted multi-linear regression and log-transformation	24
5.3	Lasso regression	29
5.4	Ridge regression	30
5.5	Elastic net	31
6	Discussion and extension	32
6.1	Discussion of the Multivariate Linear Regression model	32
6.2	Extension and Comparison	32
6.3	Decision tree	33
6.4	Gradient Boosting	34
6.5	Support Vector Machine	35
7	Data and Code availability	36

Table of Workload Distribution

Students	Completion	Assigned tasks	Score
Tran Gia Han	20%		
Nguyen Ngoc Khoi	20%		
Dinh Gia Kiet	20%		
Le Thi Phuong Thao	20%		
Bui Thai An	20%		



1 Data Introduction

The assignment focuses on analyzing a dataset which provide a comprehensive collection of information on various laptops. By delving into this dataset, we aim to provide valuable insights into the ways in which component specifications contribute to the overall cost of a laptop

1.1 Data Overview

This dataset comprises of many laptop characteristic. The following table show a brief summary of our dataset:

Properties	Description
◦ Data Source:	laptops.csv
◦ Acknowledgements:	This dataset was collected using Power Automate, more info on : https://www.kaggle.com/datasets/juanmerinobermejo/laptops-price-dataset
◦ Contents:	This dataset contains detailed specifications on various laptops .
◦ Population:	Collections of information on various laptops, enabling a detailed analysis of their specifications and pricing.
◦ Number of Variables:	12
◦ Number of Observation:	2160

Table 1: Dataset Overview

1.2 Features Overview

Continuous:

1. **RAM:** Random Access Memory - The amount of memory available for multitasking.
2. **Storage:** The storage type (HDD, SDD) and capacity of the laptop.
3. **Screen.**
4. **Final Price:** The cost of the laptop in the respective currency.

Categorical:

1. **Laptop:** The unique identifier or model name of the laptop.
2. **Status.**
3. **Brand:** Laptop brand.
4. **Model:** Laptop brand model.
5. **CPU:** The processor brand, model, and other relevant details.
6. **Storage type:** HDD, SDD.



7. **GPU:** Graphics Processing Unit - The graphics card brand, model, and associated specifications.

In this assignment, we totally use R and R Studio as tools and working environment to analyze data.

2 Background

In this chapter, we will provide a concise summary of the theories and models utilized in the project. It outlines the key theoretical frameworks and modeling approaches employed to analyze the data and achieve the project objectives.

2.1 Regression models

A regression model provides a function that describes the relationship between one or more independent variables and a response, dependent, or target variable.

2.1.1 Multiple Linear Regression (MLR)

Multiple regression indicates that there are more than one input variables that may affect the outcome, or target variable.

Definition

Multiple linear regression (MLR) is a statistical technique that is an extension of simple linear regression analysis, used to assess the association between two or more independent variables and a single continuous dependent variable.

Mathematical Equation

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Where:

- Y : Dependent variable (target variable).
- x_i : Represents the i^{th} - independent variable (input variable).
- β_0 : Intercept (value of Y when $x = 0$).
- β_i is the coefficient of each x_i .
- ε is the independent error term for the model.

Assumptions of Multiple Regression

1. **Linearity:** A linear relationship exists between the dependent and independent variables.
2. **Independence:** Observations are independent of each other.
3. **No multicollinearity:** Independent variables aren't too highly correlated with each other.
4. **Homoscedasticity:** Constant variance of the errors.
5. **No Autocorrelation:** The residuals (errors) are independent.
6. **Normality:** The residuals (the differences between observed and predicted values) of the dependent variable is normally distributed for any fixed value of the independent variables.

As the number of predictors increase in either regression model, the input-output relationship is not always straightforward, and the model can become too complex. In fact, for Lasso and Ridge regression to work in R using `cv.glmnet()` function, we need to transform categorical variables into dummy variables. In this project, we will use one-hot encoding method. The degree of the model (which is the number of predictors) increases from 7 to 84. If the model is too complex and consist of too many variables, overfitting may occur. Overfitting is the case where the model follows too strictly on the given training data due to large number of predictors, thus cannot adapt to generally new, unseen data. Therefore, we will use a technique called Regularization that takes the suitability of the model to a higher level. Regularization's role is to avoid the model being underfitting and overfitting by adding a new variable (a penalty/regularization term) while shrink predictor coefficient values in the predictive model. That is the reason it is also called "coefficient shrinkage" technique.

There are three main forms of regularization for regression models, their penalty terms' strength can be controlled by the regularization parameter `lambda`. Normally we want `lambda` to be as small as possible for the regularized loss function to be minimized, and the regularization to have significant effect to the model. We will delve into each form of regularization.

VIF: A variance inflation factor (VIF) is a measure of the amount of multicollinearity in regression analysis. Multicollinearity exists when there is a correlation between multiple independent variables in a multiple regression model.

The formula for VIF is:

$$VIF_i = \frac{1}{1 - R_i^2}$$

where: R_i^2 : Unadjusted coefficient of determination for regressing the i th independent variable on the remaining ones.

GVIF: Generalized Variance Inflation Factor (GVIF) computes the generalized variance inflation factor (GVIF) for a generalized linear model. GVIF is an extension of VIF and is used in more complex regression models where the independent variables can be numeric, categorical or more complex structures.

For categorical variables (especially non-binary ones), we will rely on

$$GVIF = VIF^{1/2Df}$$

(Scale Generalized VIF, where Df is the degree of freedom of the variable) to make GVIFs comparable across dimensions. If a categorical variable has n classes, the Df value will be $n - 1$.

ANOVA

Analysis of variance (ANOVA) is a statistical test used to evaluate the difference between the means of more than two groups. This statistical analysis tool separates the total variability within a data set into two components: random and systematic factors.

- The F-statistic is the ratio of the mean squares treatment to the mean squares error:

F-statistic: Mean Squares Treatment / Mean Squares Error

- p-value determines if the difference between group means is statistically significant.

If this p-value is less than $\alpha = 0.05$, we reject the null hypothesis of the ANOVA and conclude that there is a statistically significant difference between the means of the three groups.

if the p-value is not less than $\alpha = 0.05$ then we fail to reject the null hypothesis and conclude that we do not have sufficient evidence to say that there is a statistically significant difference between the means of the three groups.

2.1.2 Lasso Regression

Least absolute shrinkage and selection operator Regression, or Lasso regression, or L1 regularization, is a regularization technique that penalizes high-value, correlated coefficients. It introduces a regularization term (also called, penalty term) into the model's residualsum of squared loss function, and is calculated by the absolute value of the sum of coefficients. Controlled in turn by the hyperparameter `lambda` (λ), it reduces select feature weights to zero. Lasso regression thereby removes multicollinear features from the model altogether. This form of regularization determines how much you want to penalize large coefficients. This particular type of regression is well-suited for

models showing high levels of multicollinearity or when you want to automate certain parts of model selection, like variable selection/parameter elimination. If one is certain that some regressors are irrelevant, but does not know which one, Lasso regression is a suitable choice.

The goal of the algorithm is to minimize:

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda (\text{sum of the absolute value of coefficients})$$

where,

- $\lambda = 0$ implies all features are considered and it is equivalent to the linear regression where only the residual sum of squares is considered to build a predictive model,
- $\lambda = \infty$ implies no feature is considered (eliminate more and more features).

The loss function is the first term, the regularization term is the second term. The bias increases while variance decreases with increase in λ .

There is another version of Lasso Regression: Group Lasso. If we have categorical variables encoded into dummies to include in the model and want to remove the whole variables (group of coefficients) from the model, we can use Group Lasso to penalize the whole group simultaneously. However, we will stick with the traditional Lasso Regression model.

2.1.3 Ridge Regression

Ridge regression (or L2 regularization) is regularization technique that similarly penalizes high-value coefficients by introducing a penalty term in the SSR loss function. The penalty term in Ridge regression is the squared sum of coefficients. Ridge regression does not enact feature selection like Lasso Regression. While Lasso regression's penalty term can remove features from the model by shrinking coefficient values to zero, ridge regression only shrinks feature weights towards zero but never to zero. Ridge regression tends to spread coefficient values out more equally. For correlated features, it means that they tend to get similar coefficients. If one believes every regressor is relevant and is highly correlated, Ridge regression is preferred.

The goal of the algorithm is to minimize:

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p (\beta_j)^2 = \text{RSS} + \lambda (\text{sum of square of coefficients})$$

2.1.4 Elastic Net

Elastic net regularization essentially combines both ridge and lasso regression but inserting both the L1 and L2 penalty terms into the SSE loss function. L2 and L1 derive their penalty term value, respectively, by squaring or taking the absolute value of the sum of the feature weights. Elastic net inserts both of these penalty values into the cost function (SSE) equation. In this way, elastic net addresses multicollinearity while also enabling feature selection. Elastic net is mostly preferred over lasso & ridge regression because it solves the limitations of both methods, and we are usually not equipped with enough prior domain knowledge. To find the best alpha and lambda, we will perform a nested cross-validation by transforming the regularization parameter to a tuning grid.

The goal of the algorithm is to minimize:

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \left(\frac{1-\alpha}{2} \sum_{j=1}^p \beta_j^2 + \alpha \sum_{j=1}^p |\beta_j| \right)$$

2.2 Reference models

2.2.1 Decision tree

Decision tree is a popular machine learning algorithm that can be used for both regression and classification tasks. It is a non-parametric supervised learning algorithm for classification and regression tasks. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome. A decision tree can contain categorical data (YES/NO) as well as numeric data.

The basic idea behind any decision tree algorithm is as follows:

- Select the best attribute using Attribute Selection Measures (ASM) to split the records.
- Make that attribute a decision node and breaks the dataset into smaller subsets.
- All the tuples belong to the same attribute value.
- There are no more remaining attributes.
- There are no more instances.

While implementing decision tree, the main issue is how to select the best attribute for the root node and for sub-nodes. Therefore, to solve such problems there is a technique which is called as Attribute selection measure or ASM. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- Information Gain
- Gini index

1. Information Gain:

Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute. The formula for Entropy is:

$$E(S) = -p_{(+)} \cdot \log p_{(+)} - p_{(-)} \cdot \log p_{(-)}$$

Here:

- $p_{(+)}$ is the probability of positive class
- $p_{(-)}$ is the probability of negative class
- S is the subset of the training example

In decision tree, the output is mostly "Yes" or "No". Therefore, decision tree algorithm use entropy to measure the impurity of a node (Impurity is the degree of randomness). Entropy values can fall between 0 and 1. If all samples in data set, S , belong to one class, then entropy will equal zero. If half of the samples are classified as one class and the other half are in another class, entropy will be at its highest at 1.

2. Gini Index:

Gini Index or Gini impurity measures the degree or probability of a particular variable being wrongly classified when it is randomly chosen.

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

where p_i is the probability of an object being classified to a particular class.

Types of Decision Trees:

Types of decision trees are based on the type of target variable we have, which can be divided of two types:

- Categorical Variable Decision Tree: Decision Tree which has a categorical target variable then it called a Categorical variable decision tree.
- Continuous Variable Decision Tree: Decision Tree has a continuous target variable then it is called Continuous Variable Decision Tree.

Advantages of the Decision Tree:

- Decision tree is simple to understand because it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree:

The decision tree contains lots of layers, which makes it complex. It may have an overfitting issue, which can be resolved using the Random Forest algorithm. For more class labels, the computational complexity of the decision tree may increase.

2.2.2 Gradient Boosting

Gradient boosting is a machine learning technique based on boosting in a functional space, where the target is pseudo-residuals rather than the typical residuals used in traditional boosting.

There is an important parameter used in this technique known as Shrinkage. Shrinkage refers to the fact that the prediction of each tree in the ensemble is shrunk after it is multiplied by the learning rate (eta) which ranges between 0 to 1.

Gradient boosting machines (GBMs) have three main components:

- Loss function: Measures the difference between predicted and actual values.
- Base (or “weak”) learners: Decision trees built sequentially, each focusing on correcting the errors made by the previous tree.
- Additive model: Combines the predictions of all base learners to produce the final prediction.

Gradient boosting is iterative. Each iteration invokes the following formula:

$$F_{i+1} = F_i - f_i$$

where:

- F_i is the strong model at step i , with initially $F_0(x) = 0$
- f_i is the weak model at step i

2.2.3 Support Vector Machine

A Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding a hyperplane in a high-dimensional space that best separates data into different classes, which aims to maximize the margin (the distance between the hyperplane and the nearest data points of each class) while minimizing classification errors.

Types of SVM classifiers:

- Linear SVMs: Linear SVMs are used with linearly separable data, which means that the data do not need to undergo any transformations to separate the data into different classes. This separating hyperplane can be represented as: $wx + b = 0$, where w is the weight vector, x is the input vector, and b is the bias term.
- Kernel or non-linear SVM: Non-linear data that cannot be segregated into distinct categories with the help of a straight line is classified using a kernel or non-linear SVM. Here, the classifier is referred to as a non-linear classifier. The classification can be performed with a non-linear data type by adding features into higher dimensions rather than relying on 2D space.

3 Data pre-processing

In this section, we applied R studio for the essential steps of data pre-processing, an important phase that involves a series of carefully orchestrated transformations.

3.1 Data reading

3.1.1 Libraries

We import necessary libraries for later use:



```
1 library(corrplot)
2 library(caTools)
3 library(ggplot2)
4 library(dplyr)
5 library(ggpubr)
6 library(caret)
7 library(lattice)
8 library(glmnet)
9 library(rpart)
10 library(Metrics)
11 library(readxl)
12 library(car)
13 library(boot)
14 library(e1071)
15 library(gbm)
```

3.1.2 Importing data

```
1 data <- read.csv("laptops.csv")
2 View(data)
3 str(data)
```

	Laptop	Status	Brand	Model	CPU	RAM	Storage	Storage.type	GPU	Screen	Touch	Final.Price
1	ASUS ExpertBook B1 B1502CBA-EJ0436X Intel Core i5-1235...	New	Asus	ExpertBook	Intel Core i5	8	512	SSD		15.6	No	1009.00
2	Alurin Go Start Intel Celeron N4020/8GB/256GB SSD/15.6"	New	Alurin	Go	Intel Celeron	8	256	SSD		15.6	No	299.00
3	ASUS ExpertBook B1 B1502CBA-EJ0424X Intel Core i3-1215...	New	Asus	ExpertBook	Intel Core i3	8	256	SSD		15.6	No	789.00
4	MSI Katana GF66 12UC-082XES Intel Core i7-12700H/16GB/...	New	MSI	Katana	Intel Core i7	16	1000	SSD	RTX 3050	15.6	No	1199.00
5	HP 15S-FQ508SNS Intel Core i5-1235U/16GB/512GB SSD/1...	New	HP	15S	Intel Core i5	16	512	SSD		15.6	No	669.01
6	MSI Crosshair 17 C12VF-264XES Intel Core i7-12650H/32GB...	New	MSI	Crosshair	Intel Core i7	32	1000	SSD	RTX 4060	17.3	No	1699.00
7	Lenovo Thinkpad E14 Gen 4 AMD Ryzen 5 5625U/8GB/256G...	New	Lenovo	ThinkPad	AMD Ryzen 5	8	256	SSD		14.0	No	909.00
8	ASUS VivoBook 15 F515JA-EJ2882W Intel Core i7-1065G7/8...	New	Asus	VivoBook	Intel Core i7	8	512	SSD		15.6	No	809.01
9	Medion Akoya E15415 Intel Core i5-10210U/8GB/256GB SS...	New	Medion	Akoya	Intel Core i5	8	256	SSD		15.6	No	519.00
10	HP Victus 16-d1038ns Intel Core i7-12700H/16GB/512GB SS...	New	HP	Victus	Intel Core i7	16	512	SSD	RTX 3050	16.1	No	1149.00
11	Lenovo V15 IGL Intel Celeron N4020/8GB/256 GB SSD/15.6"	New	Lenovo	V15	Intel Celeron	8	256	SSD		15.6	No	349.00
12	MSI Thin GF63 12VE-021XES Intel Core i7-12650H/16GB/1T...	New	MSI	Thin	Intel Core i7	16	1000	SSD	RTX 4050	15.6	No	1399.00
13	ASUS ROG Strix G15 G513RC-HN088 AMD Ryzen 7 6800H/1...	New	Asus	ROG	AMD Ryzen 7	16	512	SSD	RTX 3050	15.6	No	1199.00
14	Lenovo V15 G3 ABA AMD Ryzen 5 5625U/16GB/512GB SSD...	New	Lenovo	V15	AMD Ryzen 5	16	512	SSD		15.6	No	476.99
15	Lenovo IdeaPad 1 15ADA7 AMD Ryzen 3 3250U/8GB/256GB...	New	Lenovo	IdeaPad	AMD Ryzen 3	8	256	SSD		15.6	No	391.00
16	HP 15S-FQ5013NS Intel Core i5-1235U/8GB/512GB SSD/15.6"	New	HP	15S	Intel Core i5	8	512	SSD		15.6	No	699.00
17	Lenovo IdeaPad 3 15IAU7 Intel Core i5-1235U/16GB/512GB ...	New	Lenovo	IdeaPad	Intel Core i5	16	512	SSD		15.6	No	799.00
18	HP 15S-FQ2163NS Intel Core i3-1115G4/8GB/512GB SSD/1...	New	HP	15S	Intel Core i3	8	512	SSD		15.6	No	549.00
19	ASUS VivoBook 15 F515JA-EJ4134 Intel Core i7-1065G7/8G...	New	Asus	VivoBook	Intel Core i7	8	512	SSD		15.6	No	769.00
20	MSI GF63 Thin 11UC-446XES Intel Core i7-11800H/16GB/51...	New	MSI	Thin	Intel Core i7	16	512	SSD	RTX 3050	15.6	No	999.00

Figure 1: Data reading

`str()` command in R is used to print out the structure of the data object, which provides information about the data type of each column in the dataframe. In the raw data, "Status", "Brand", "Model", "CPU", "Storage.type", "GPU", "Touch" characterized as character variables and are also categorical variables.

3.2 Data analysis

3.2.1 Check missing value

```
1 summary(data)
2 sum(is.na(data))
3 colSums(is.na(data))
```

```
> summary(data)
Laptop      Status      Brand      Model      CPU      RAM      Storage      Storage.type      GPU      Screen
Length:2160  Length:2160  Length:2160  Length:2160  Length:2160  Min.   : 4.00  Min.   : 0.0  Length:2160  Length:2160  Min.   :10.10
Class :character  Class :character  Class :character  Class :character  Class :character  1st Qu.: 8.00  1st Qu.: 256.0  Class :character  Class :character  1st Qu.:14.00
Mode :character    Mode :character    Mode :character    Mode :character    Mode :character    Median :16.00  Median : 512.0  Mode :character  Mode :character  Median :15.60
                                          Mean  :15.41  Mean  : 596.3                                     Mean :15.17
                                          3rd Qu.:16.00  3rd Qu.:1000.0                                   3rd Qu.:15.60
                                          Max.   :128.00  Max.   :4000.0                                   Max.   :18.00
                                          NA's   :4
                                          NA's   :4

Touch      Final.Price
Length:2160  Min.   : 201.1
Class :character  1st Qu.: 661.1
Mode :character    Median :1031.9
                    Mean  :1312.6
                    3rd Qu.:1709.0
                    Max.   :7150.5

> sum(is.na(data))
[1] 4
> colSums(is.na(data))
Laptop      Status      Brand      Model      CPU      RAM      Storage      Storage.type      GPU      Screen      Touch      Final.Price
0          0          0          0          0          0          0          0          0          4          0          0
```

Figure 2: Missing values

There are 4 rows containing the missing values in "Screen" column and "Laptop" column just includes the name of laptop, does not effect significantly to the price prediction.

When we use the str() function to examine the structure of the dataframe, we observe that the dataframe comprises two columns: Model and Brand. Instead of using each column separately, we decide to merge these two columns together to create a new column containing the specific name of each laptop.

```
1 data$Model <- paste(data$Brand, data$Model, sep = " ")
```

The columns "GPU" and "Storage.type" contain null values that are not defined as NA. To address this issue, we first calculate the number of rows in each column. After examining the data, we decide to change the corresponding rows in the "Storage.type" column to "SSD" ("SSD" is the most frequent value, occurring 2062 times) while keeping these rows unchanged in the "GPU" column.

To manage outliers during the modeling process, we remove variables (except "Final.price") that appear fewer than 5 times in the dataset, which may not contribute significantly to the modeling process.

```
1 # Replace null value in column "Storage.type"
2 data$Storage.type <- ifelse(data$Storage.type == "", "SSD", data$Storage.type)
3 data$GPU <- ifelse(data$GPU == "", "No GPU", data$GPU)
4
5 data <- na.omit(data)
6 # Remove attribute "Laptop" and "Brand"
7 data <- data[, -which(names(data) == "Laptop")]
8 data <- data[, -which(names(data) == "Brand")]
9 # Remove outliers
10 variables <- setdiff(names(data), "Final.Price")
11 for (variable in variables) {
12   value_counts <- table(data[[variable]])
13   values_to_remove <- names(value_counts[value_counts <= 5])
14   data <- data[!(data[[variable]] %in% values_to_remove), ] }
```

3.2.2 Data statistic

R factors: Factors in R Programming Language are data structures that are implemented to categorize the data or represent categorical data and store it on multiple level.

```
1 data$CPU <- as.factor(data$CPU)
2 data$Status <- as.factor(data$Status)
3 data$Model <- as.factor(data$Model)
4 data$GPU <- as.factor(data$GPU)
5 data$Touch <- as.factor(data$Touch)
6 data$Storage.type <- as.factor(data$Storage.type)
```

We use **factor()** to convert "Status", "Brand", "Model", "CPU", "Storage.type", "GPU", "Touch" column in the dataframe to factor. This helps R understand these columns have categorical data.

3.3 Correlation coefficients between variables

Correlation refers to the relationship between two variables, which refers to the degree of linear correlation between any two random variables. This correlation matrix in R can be expressed as a range of values expressed within the interval $[-1, 1]$.

In this dataset, we choose 4 columns: "RAM", "Storage", "Screen" and "Final.Price" to create correlation matrix since other columns are categorical variables, their correlation with them may not be meaningful.

```
> cor_mat
      RAM Storage Screen Final.Price
RAM    1.000000 0.7512709 0.3614035  0.7251956
Storage 0.7512709 1.0000000 0.3980253  0.6961537
Screen  0.3614035 0.3980253 1.0000000  0.2683586
Final.Price 0.7251956 0.6961537 0.2683586  1.0000000
```

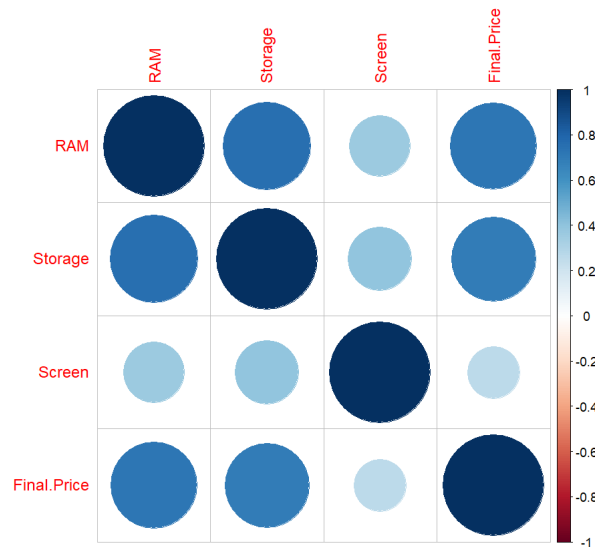


Figure 4: Correlation matrix between 4 columns

Based on the image, we observe that there is a strong correlation between the variables. Therefore, we decide to utilize the these columns to predict the "Final Price." columns.

4 Descriptive statistics

Understanding the underlying structure, trends, and distributions of the variables at hand is paramount. The most essential aspect of this section is to provide a comprehensive overview of the dataset's characteristics.

4.1 Data summary



```
> summary(data)
Status      Brand      Model      CPU      RAM      Storage      Storage.type
0: 662      4      :415      5      : 115      18      :708      Min.      : 4.00      Min.      : 0.0      0: 42
1:1494     10      :367     57      : 103     17      :534     1st Qu.: 8.00     1st Qu.: 256.0     1: 56
      13      :364     90      : 101     9       :156     Median : 16.00     Median : 512.0     2:2058
      18      :308    114      : 99     16      :130     Mean   : 15.42     Mean   : 596.6
      1       :136    102      : 98     8       :126     3rd Qu.: 16.00     3rd Qu.:1000.0
      3       :116     68      : 68     15      : 94     Max.    :128.00     Max.    :4000.0
      (Other):450   (Other):1572 (Other):408
      GPU      Screen      Touch      Final.Price
1       :1370   Min.    :10.10   0:1930   Min.    : 201.1
23      : 129   1st Qu.:14.00   1: 226   1st Qu.: 661.1
24      : 121   Median :15.60           Median :1031.7
25      : 97    Mean   :15.17           Mean   :1312.1
28      : 62    3rd Qu.:15.60           3rd Qu.:1704.5
26      : 51    Max.    :18.00           Max.    :7150.5
(Other): 326
```

Figure 5: Summary of data

First, we will display the overview of data using `summary(data)`

For our discrete variable or categorical variables, we used `as.factor()` as mention above to convert a variable or vector to a factor for later use. Then display to terminal summary of these variables.

```
> xtabs(~RAM,data = data)
RAM
 4   6   8  12  16  32  40  64 128
68   3 816  15 925 301   2  25   1

> xtabs(~Storage.type + Storage,data = data)
Storage
Storage.type  0  32  64 128 240 256 500 512 1000 2000 3000 4000
0            1   4   3   8   0  13   5   6   2   0   0   0
1            0   9  32  15   0   0   0   0   0   0   0   0
2            0   1   0  44   1 436  32 932  568  42   1   1
```

```
> xtabs(~Brand,data = data)
Brand
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27
136 29 116 415   8  84   1  19  48 367   6   1 364  32  32  77   2 308  24   8   3  37   1  22   4   1  11

> xtabs(~GPU,data = data)
GPU
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22
1370  2   1   1   3   1  50  14   1   5   7   5   1   2   9   3   1  10  20  25   7   1
 23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40  41  42  43  44
129 121  97  51  33  62  40  18  17   6   7   7   3   3   1   1   2   1   6   1   5   4
 45
  2

> xtabs(~Touch + Screen, data = data)
Screen
Touch 10.1 10.5 10.95 11.6 12 12.3 12.4 12.5 13 13.3 13.4 13.5 13.6 13.9 14 14.1 14.2 14.4 14.5 15 15.3 15.4
  0   0   0   0   9   4   8   0   1   8 105   2   3  11   0 339  11  14   0   3   7   8   5
  1   1   7   1   7   0   5   6   0 19  26  17  16   0   2  53   0   0   6   0 14   0   0

Screen
Touch 15.6 16 16.1 16.2 17 17.3 18
  0 998 151  48  10  22 160   3
  1  11  23   0   0  11   1   0
```

Figure 1: Summary for discrete and categorical variables

4.2 Data plot

We plot the histogram and summarise Final.Price

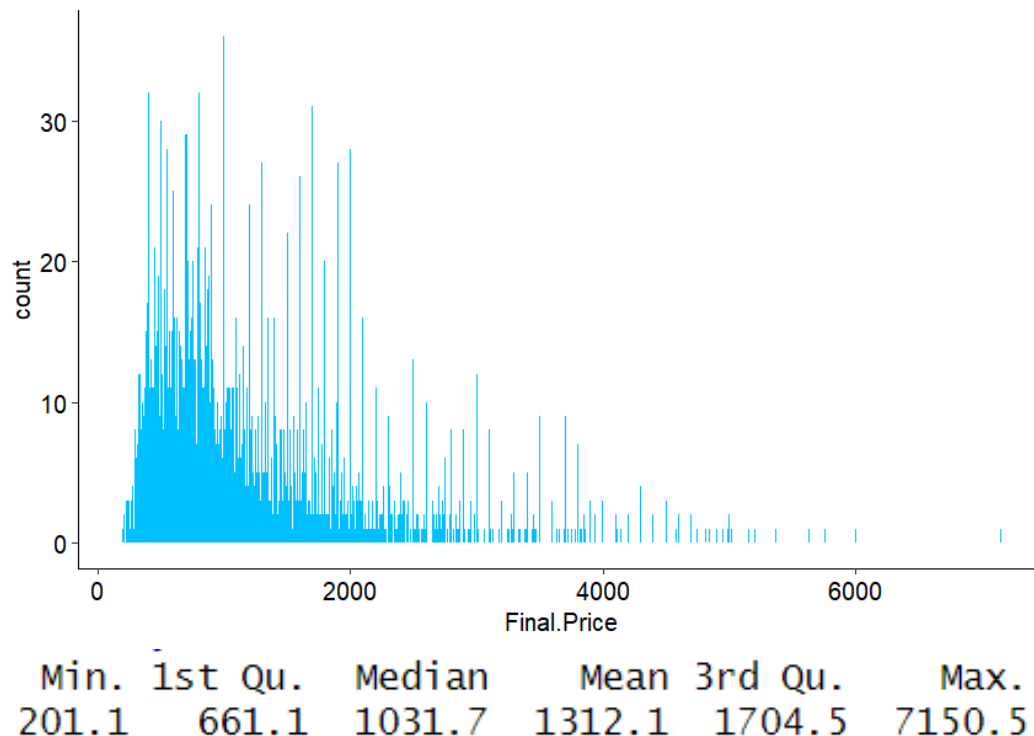


Figure 3: Histogram and Summary of Final.Price

R studio code:

```
1 data %>%
2   ggplot(aes(x=Final.Price))+
3   geom_histogram(binwidth = 10, fill = "deepskyblue")
```

```
summary(data$Final.Price)
```

We can witness from the histogram that: The Final.Price range from 201.1 to 7150.5, most of the observe objects have the price less than 3000. Only a few models have the price higher than 6000, and we can treat them as outliers.

We continue plot some graphs to analyze the factors that affect the Final.Price

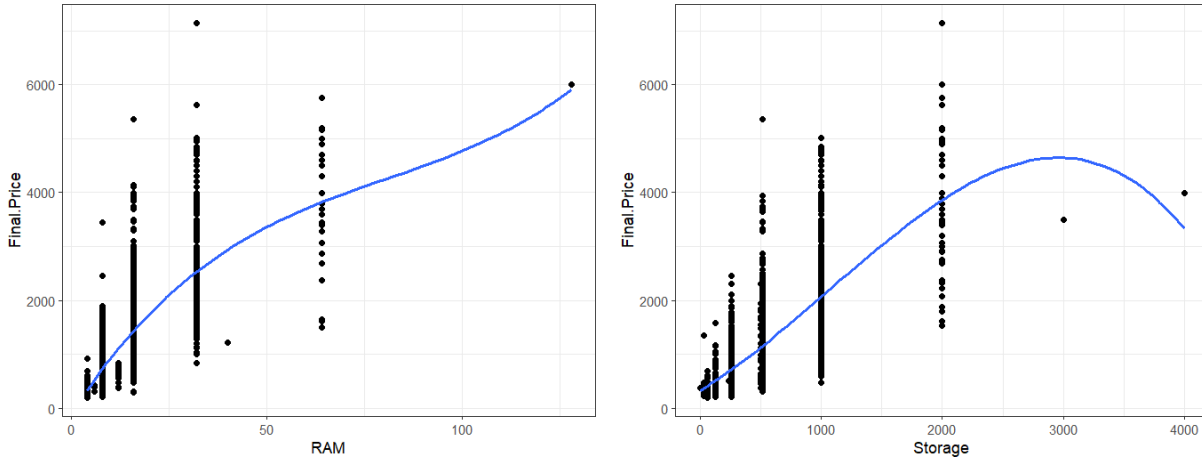


Figure 4: RAM and Storage scatter plot

R studio code:

```
1 data%>%
2 ggplot(aes(RAM, Final.Price)) +
3 geom_point() +
4 geom_smooth(method = lm, formula = y ~ splines::bs(x, 3), se = FALSE) +
5 theme_bw()
6 data%>%
7 ggplot(aes(x=Storage, y = Final.Price))+
8 geom_point() +
9 geom_smooth(method = lm, formula = y ~ splines::bs(x, 3), se = FALSE) +
10 theme_bw()
```

The scatter plots shows a positive correlation between both storage and ram capacity with the final price. When the value of RAM or Storage increase, Final.Price tends to rise as well. This suggest that higher RAM or Storage command a premium.

We further analyze the data since there are three types of storage: 0 is no storage, 1 is "eMMC" and 2 is "SSD".

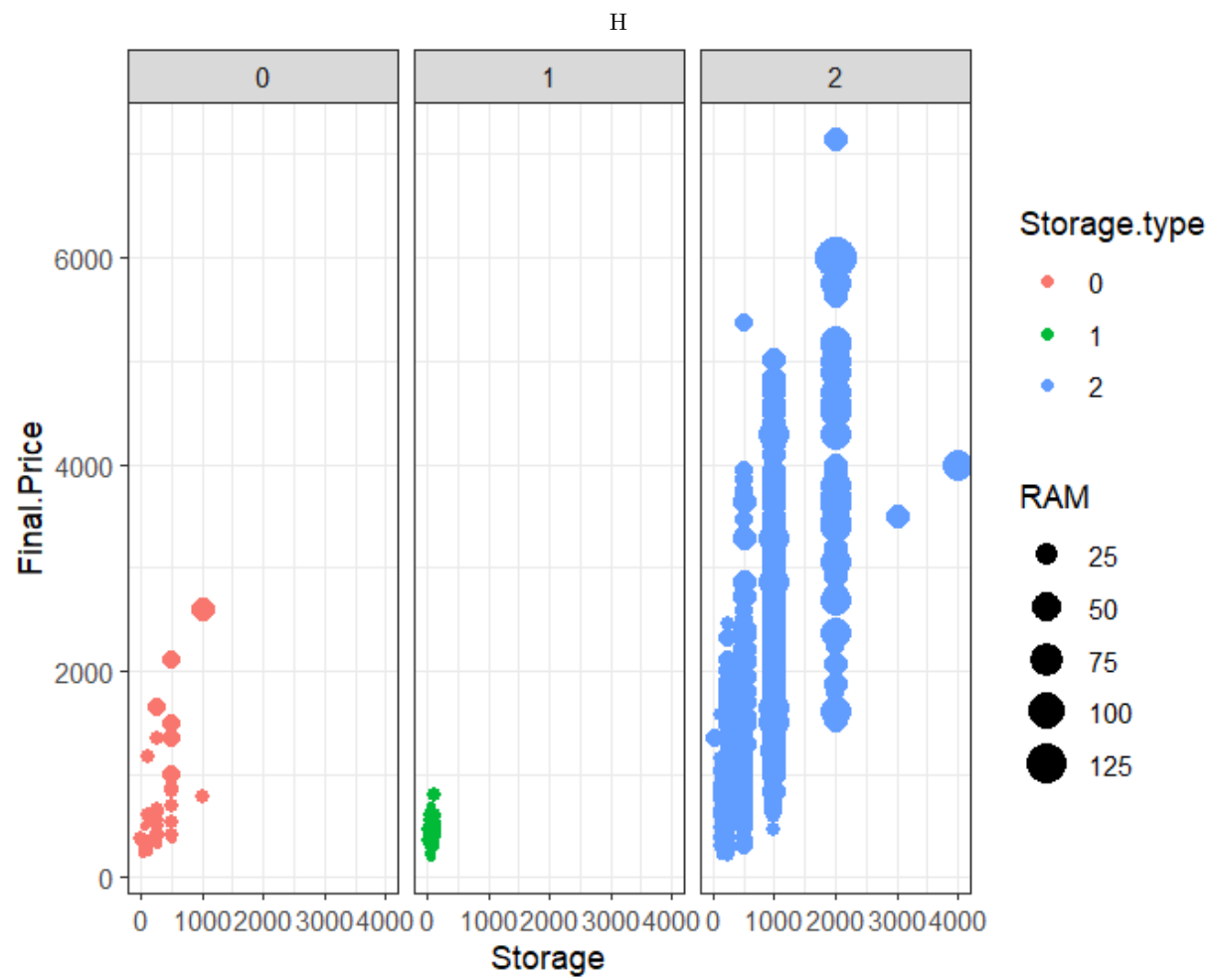
R code:

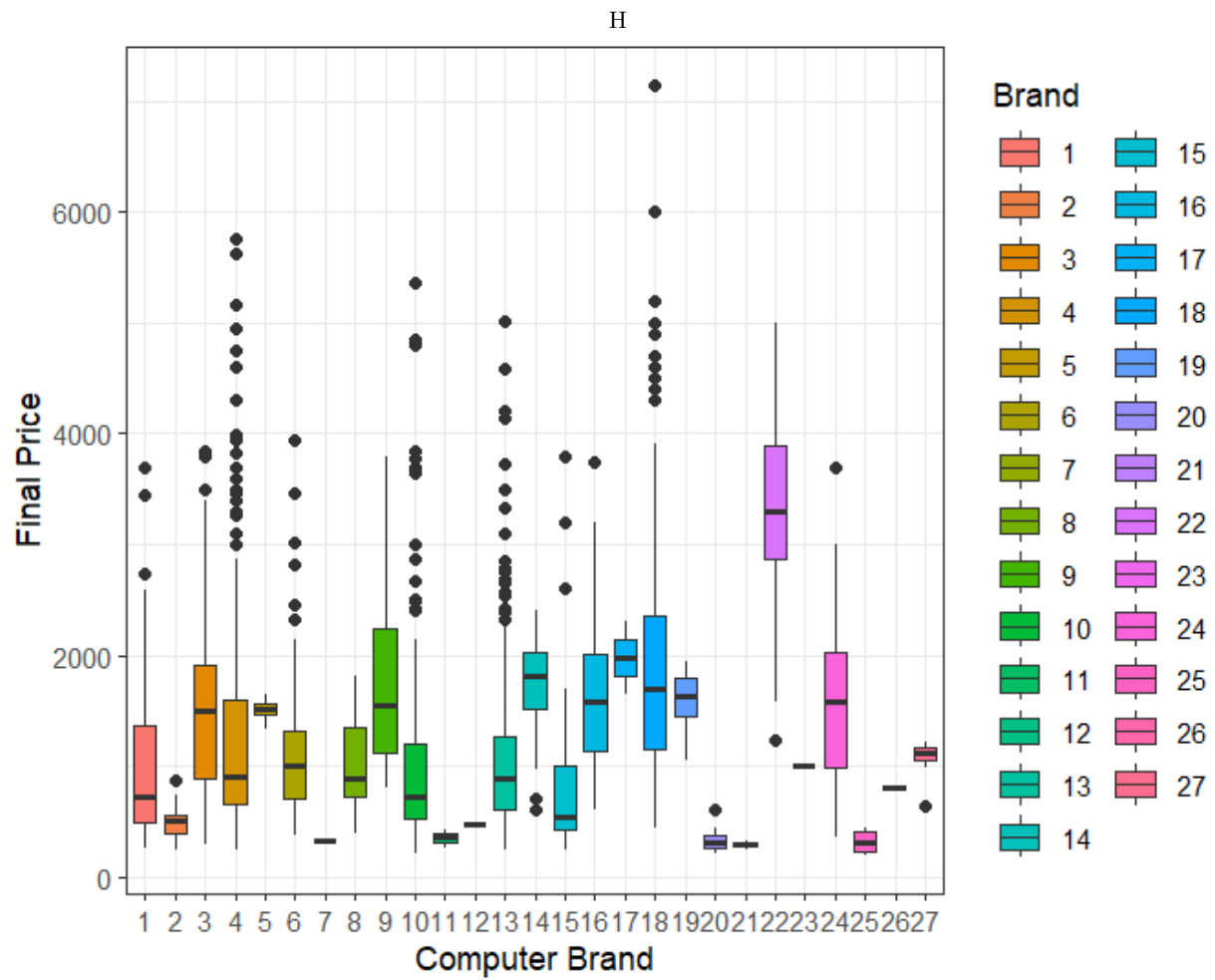
```
1 data%>%
2 + ggplot(aes(x=Storage, y = Final.Price,
3 color = Storage.type))
4 + geom_point(aes(size = RAM))
5 + facet_wrap(~Storage.type)+theme_bw()
```

Models have no storage have the Final.Price less than 3000. Meanwhile, computers with eMMC storage type witnessed a small range in both Final.price (less than 1000) and Storage size (less than 256). In contrast, SSD storage spans a wider price range and dominates the chart.

We use boxplot to illustrate the median, identify the interquartile range, detect skewness, identify outliers and compare distributions of each laptop brands final price.

R code:





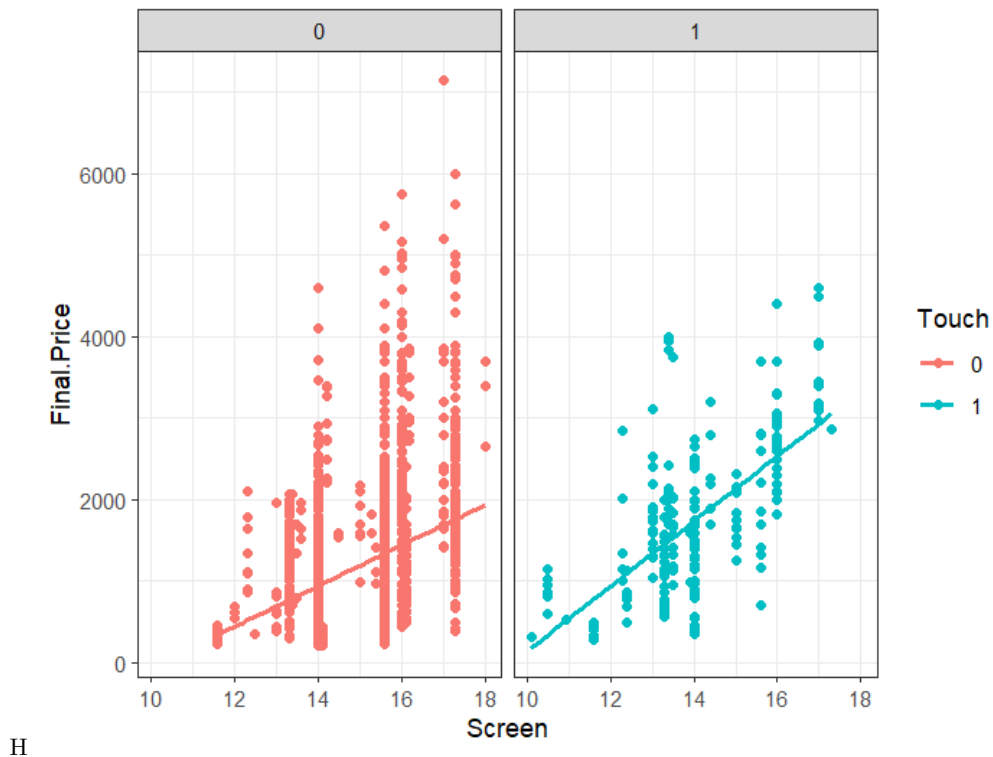


Figure 2: Screen size scatter plot

```
1 data%>%
2 ggplot(aes(x = Brand,y = Final.Price)) +
3 geom_boxplot(position = "dodge",aes(fill = Brand))+
4 theme_bw()+
5 labs(x = "Computer Brand", y = "Final Price")
```

The chart provided a boxplot visualization that compares the final prices of various computer brands. Brands 3, 4, and 5 have lower median prices, as indicated by the lower positioning of their boxes on the chart. Brand 17 stands out with one of the highest median prices. Several brands have outliers, which are data points that deviate significantly from the rest of the data, shown as dots above or below the boxes.

Finally, we illustrates the relationship between screen size and final price of devices, with a distinction made between devices with and without touch capabilities.

R code:

```
1 data%>%
2 ggplot(aes(x = Screen,y = Final.Price,colour = Touch))+
3 geom_point()+
4 geom_smooth(method = lm,se = FALSE)+
5 facet_wrap(~Touch)+
6 theme_bw()
```

There is an upward trend in price as screen size increases for both touch and non-touch devices, with touch devices showing a more consistent increase. However, devices with touch capability (indicated by '1') are generally priced higher than non-touch devices (indicated by '0'). The trend lines serve as a visual representation of these relationships within the dataset.

5 Inferential statistics

5.1 Multi-linear Regression

Firstly, we try predicting the price of laptops (dependent variable) using all predictors (independent variables) in the dataset.

A critical step in the machine learning workflow is the segregation of data into distinct training and testing sets. This partitioning strategy serves to mitigate the risk of overfitting. We will split the data into two subdata: The training set constitutes the majority of the data and is employed to construct the model, and the remaining data forms the testing set, which is reserved for evaluating the model's generalizability on unseen examples. This approach fosters the development of robust models that can effectively perform on new data, enhancing their overall reliability.

```
1 # Split data with ratio train:test = 7:3
2 train_indices_full <- createDataPartition(data$Final.Price, p = 0.7, list = FALSE)
3 train_data_full <- data[train_indices_full, ]
4 test_data_full <- data[-train_indices_full, ]
5
6 # Train a multilinear regression model
7 MLR_full <- lm(Final.Price ~ . - Final.Price, data = train_data_full)
8
9 # Summarize the results
10 print(summary(MLR_full))
```

Listing 1: Data Splitting and Training

Here are the resulting details of our model:

```
1 ----- Multi-linear regression with full predictors -----
2
3 Call:
4 lm(formula = Final.Price ~ . - Final.Price, data = train_data_full)
5
6 Residuals:
7      Min       1Q   Median       3Q      Max
8 -1177.94  -149.83   -26.49   130.77  2079.29
9
10 Coefficients:
11              Estimate Std. Error t value Pr(>|t|)
12 (Intercept)    -102.64805    236.97067   -0.433  0.664976
13 StatusRefurbished    -264.06748     24.09089  -10.961 < 2e-16 ***
14 ModelAcer Chromebook    221.77159    161.62153    1.372  0.170284
15 ModelAcer ConceptD     641.74575    154.01843    4.167  3.33e-05 ***
16 ModelAcer Extensa    -123.98791     97.58838   -1.271  0.204161
17 ModelAcer Nitro    -139.97581    111.18043   -1.259  0.208291
18 ModelAcer Predator    -46.65592    125.31746   -0.372  0.709738
19 ModelAlurin Flex Advance -147.62770    112.96171   -1.307  0.191518
20 ModelApple MacBook Air   457.70687     98.77900    4.634  4.01e-06 ***
21 ModelApple MacBook Pro   596.04475     98.17280    6.071  1.73e-09 ***
22 ModelAsus Chromebook    161.40551    138.10108    1.169  0.242751
23 ModelAsus ExpertBook    152.24154     80.14511    1.900  0.057744 .
24 ModelAsus F515         33.97841     99.27689    0.342  0.732220
25 ModelAsus ROG          98.34929     96.15233    1.023  0.306598
26 ModelAsus TUF         -95.00706     95.34069   -0.997  0.319220
27 ModelAsus VivoBook     -50.85990     69.87958   -0.728  0.466873
28 ModelAsus ZenBook      196.12672     88.05716    2.227  0.026126 *
29 ModelDell Latitude     452.79143     83.82143    5.402  8.04e-08 ***
30 ModelDell Vostro       185.01984     86.41947    2.141  0.032491 *
31 ModelGigabyte Aero      199.09865    131.04207    1.519  0.128954
32 ModelGigabyte Aorus     80.82630    124.20736    0.651  0.515349
33 ModelHP 15S          -104.20710     68.56869   -1.520  0.128854
34 ModelHP 250         -74.84932     89.48536   -0.836  0.403083
35 ModelHP 255          54.06192    110.35158    0.490  0.624295
36 ModelHP Chromebook     54.55724    208.12774    0.262  0.793266
37 ModelHP EliteBook      568.13531     83.51814    6.803  1.66e-11 ***
38 ModelHP Omen         -187.14247    106.70203   -1.754  0.079722 .
39 ModelHP Pavilion       87.04940    114.69129    0.759  0.448018
```



40	ModelHP ProBook	292.35545	90.78581	3.220	0.001317	**
41	ModelHP Victus	-65.15523	104.22053	-0.625	0.531988	
42	ModelHP Zbook	1755.76674	149.79982	11.721	< 2e-16	***
43	ModelLenovo IdeaPad	-14.97663	69.76009	-0.215	0.830050	
44	ModelLenovo Legion	332.81907	111.31548	2.990	0.002851	**
45	ModelLenovo ThinkBook	368.23980	79.56841	4.628	4.12e-06	***
46	ModelLenovo ThinkPad	551.66169	72.90164	7.567	7.88e-14	***
47	ModelLenovo V15	-90.92614	75.85702	-1.199	0.230915	
48	ModelLenovo Yoga	372.92319	106.26580	3.509	0.000467	***
49	ModelLG Gram	337.74031	110.34895	3.061	0.002261	**
50	ModelMedion Akoya	-26.70746	102.27471	-0.261	0.794036	
51	ModelMicrosoft Surface Laptop	527.09576	99.34693	5.306	1.35e-07	***
52	ModelMicrosoft Surface Pro	627.58389	103.99001	6.035	2.15e-09	***
53	ModelMSI Creator	172.00455	108.28177	1.588	0.112455	
54	ModelMSI Katana	46.76639	114.92376	0.407	0.684133	
55	ModelMSI Leopard	-105.71108	129.98900	-0.813	0.416257	
56	ModelMSI Modern	-25.24732	90.93241	-0.278	0.781332	
57	ModelMSI Prestige	146.90461	96.16434	1.528	0.126882	
58	ModelMSI Pulse	40.47192	120.59355	0.336	0.737230	
59	ModelMSI Raider	336.75412	115.58961	2.913	0.003646	**
60	ModelMSI Stealth	224.15562	109.67939	2.044	0.041211	*
61	ModelMSI Summit	105.29786	114.64056	0.919	0.358551	
62	ModelMSI Thin	-181.24898	125.47497	-1.445	0.148875	
63	ModelPcCom Revolt	71.87315	120.75307	0.595	0.551824	
64	ModelRazer Blade	1139.56391	105.80781	10.770	< 2e-16	***
65	ModelSamsung Galaxy Book	447.58692	115.36418	3.880	0.000111	***
66	CPUAMD Ryzen 3	-74.77400	109.32593	-0.684	0.494144	
67	CPUAMD Ryzen 5	-18.48843	100.44384	-0.184	0.853994	
68	CPUAMD Ryzen 7	91.10434	102.32770	0.890	0.373484	
69	CPUAMD Ryzen 9	378.39455	124.34315	3.043	0.002395	**
70	CPUApple M1	297.46873	156.03754	1.906	0.056853	.
71	CPUApple M2	618.32217	145.72867	4.243	2.39e-05	***
72	CPUIntel Celeron	-71.17002	114.06437	-0.624	0.532789	
73	CPUIntel Core i3	-40.15584	101.17542	-0.397	0.691521	
74	CPUIntel Core i5	31.38978	97.56166	0.322	0.747707	
75	CPUIntel Core i7	190.60865	100.00865	1.906	0.056914	.
76	CPUIntel Core i9	660.92066	119.03015	5.553	3.51e-08	***
77	CPUIntel Evo Core i5	63.84667	123.49704	0.517	0.605265	
78	CPUIntel Evo Core i7	479.77417	113.66223	4.221	2.63e-05	***
79	RAM	14.85087	1.90940	7.778	1.66e-14	***
80	Storage	0.51517	0.05412	9.519	< 2e-16	***
81	Storage.typeeMMC	75.22237	157.29543	0.478	0.632584	
82	Storage.typeSSD	86.81373	77.16995	1.125	0.260841	
83	GPUGTX 1650	64.62215	67.25704	0.961	0.336846	
84	GPUGTX 1660	31.03304	142.34433	0.218	0.827458	
85	GPURTX 2060	31.68641	111.73195	0.284	0.776774	
86	GPURTX 2070	327.46942	112.40253	2.913	0.003646	**
87	GPURTX 3050	403.67640	58.21009	6.935	6.83e-12	***
88	GPURTX 3060	439.42075	68.61601	6.404	2.21e-10	***
89	GPURTX 3070	681.32147	80.26742	8.488	< 2e-16	***
90	GPURTX 3080	1272.92930	92.04461	13.829	< 2e-16	***
91	GPURTX 4050	397.16604	90.87903	4.370	1.36e-05	***
92	GPURTX 4060	500.26839	84.91382	5.891	5.04e-09	***
93	GPURTX 4070	718.89770	94.57698	7.601	6.14e-14	***
94	GPURTX 4080	1695.90765	125.99126	13.461	< 2e-16	***
95	GPURTX 4090	1819.40480	140.26730	12.971	< 2e-16	***
96	Screen	20.73131	12.71528	1.630	0.103291	
97	TouchYes	344.71639	40.65839	8.478	< 2e-16	***
98	---					
99	Signif. codes: 0	***	0.001	**	0.01	*
100					0.05	.
101					0.1	
102	Residual standard error: 300.6	on	1130	degrees of freedom		
103	Multiple R-squared: 0.8851,	Adjusted R-squared: 0.8765				
104	F-statistic: 102.4	on	85	and 1130 DF, p-value: < 2.2e-16		

Listing 2: Summary of the Multilinear Regression Model

We start conducting the test for our model:

```
1 MLR_full_predictions <- predict(MLR_full, newdata = test_data_full)
2 accuracy_MLR_full <- data.frame(Actual = test_data_full$Final.Price, Predicted = MLR_full_predictions)
3
4 MAPE_MLR_full <- mape(accuracy_MLR_full$Actual, accuracy_MLR_full$Predicted)
5 cat("MAPE:", MAPE_MLR_full, "\n")
6
7 cat("train_rmse = ", summary(MLR_full)$sigma, "\ntest_rmse = ", rmse(accuracy_MLR_full$Actual, accuracy_MLR_full$Predicted), "\n")
```

Listing 3: Predict on the test dataset

The result of the prediction:

```
MAPE: 0.1986867
train_rmse = 300.5598
test_rmse = 305.5981
```

As observed:

- Residual Standard Error (RSE or RMSE) being 300.6 reflects the typical deviation between actual and predicted final prices on the training set.
- To assess the performance of the previously developed multilinear regression model in predicting final product prices, the primary metric employed is the Mean Accurate Percentage Error (MAPE). The model achieves MAPE (on the testing dataset), of 19.87%, which is fairly high, but within an acceptable range. This value indicates that, on average, the absolute deviations between predicted and actual final prices from their true values amount to 19.87% in percentage terms. In simpler terms, for a product with an actual final price of \$100, the model's prediction might deviate by an average of approximately \$19.87 (in absolute value). This MAPE value suggests that the model exhibits the ability to predict final prices within a relatively tight range compared to the actual values. This finding highlights the model's success in capturing the key factors that significantly influence final product prices.
- The **Multiple R-squared** value being 0.8851 indicates that the model explains approximately 88.51% of the variance in final price. However, it is statistically better to use the **Adjusted R-squared** value (87.65%). The reason behind this is because we are using a lot of categorical predictors that are one-hot encoded to dummy variables. While the multiple R-squared value tends to increase as more variables are added to the model (even if they don't improve the model significantly), the adjusted R-squared value penalizes the addition of unnecessary variables. This adjustment helps provide a more accurate measure of the model's goodness of fit.
- F-Statistic and F-test: In addition to looking at whether individual features have a significant effect, we may also wonder whether at least one feature has a significant effect. That is, we would like to test the null hypothesis:

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_{p-1} = 0$$

that all of the population coefficients (except the intercept) are 0 against the alternative hypothesis:

$$H_1 : \exists i : 1 \leq i \leq p - 1 : \beta_i \neq 0$$

Here the p-value for our F-test is less than $2.2e^{-16}$, indicating that the null hypothesis should be rejected, and we can conclude there is at least one predictor that is statistically significant to our model.

The next step is to eliminate some predictors from our model for better accuracy and fit. Since most of our predictors are categorical, we will perform the ANOVA test for our MLR model. Any predictor with p-value larger than 0.01 will be eliminated.

```
1 anova_MLR_full <- anova(MLR_full)
2 print(anova_MLR_full)
```

Listing 4: ANOVA of model conducted in R

```

1 ----- Performing ANOVA test -----
2 Analysis of Variance Table
3
4
5 Response: Final.Price
6      Df    Sum Sq   Mean Sq    F value    Pr(>F)
7 Status      1    193209    193209      2.1388    0.1439
8 Model     52 600858864 11554978 127.9108 <2e-16 ***
9 CPU       13  92547527   7119041   78.8061 <2e-16 ***
10 RAM       1  37202699   37202699 411.8250 <2e-16 ***
11 Storage    1 11709921 11709921 129.6260 <2e-16 ***
12 Storage.type  2    39897    19949    0.2208    0.8019
13 GPU       13 37600368   2892336   32.0175 <2e-16 ***
14 Screen     1    13422    13422    0.1486    0.7000
15 Touch      1  6493597   6493597   71.8826 <2e-16 ***
16 Residuals 1130 102079892    90336
17 ---
18 Signif. codes:  0      ***    0.001    **    0.01    *    0.05    .    0.1    1

```

Listing 5: ANOVA Result

We choose the significance level of 0.01 for the ANOVA. If a factor has $Pr(> F) > 0.01$, they are not statistically significant in the contribution of the laptop's **Final.Price**. From the summary, it is clear that the predictors **Storage.type** and **Screen** does not contribute significantly in the prediction of laptop price, with the p-value (shown as $Pr(> F)$ in the summary) at 0.8019 and 0.7, respectively. Additionally, the correlation matrix for numerical variables also show very little correlation of the independent variable **Screen** to **Final.Price**. Therefore, our MLR model need to remove **Storage.type** and **Screen** from the list of candidate predictors. The p-value of **Status** may vary with the random seed and will have a chance to pass the 0.01 threshold. Unfortunately the p-value for **Status** is 0.1439, passing the threshold for this seed. However, according to the domain knowledge of our group, the status of the laptop is considered one of the most crucial factor to decide the price of any laptop. Therefore, we will perform three Likelihood Ratio Tests (LRT) of level $\alpha = 0.01$ to further **statistically** conclude that whether or not **Status** should be included in our model.

The purpose of LRT is to double-check whether predictors **Screen** and **Storage.type** have significant effect on our model or not, while deciding if **Status** should be excluded or not. LRT calculates the ratio of two likelihood functions. Asymptotically, the test statistic is distributed as a chi-squared random variable, with degrees of freedom equal to the difference in the number of parameters between the two models.

```

1 > print(anova(MLR, MLR_full, test="LRT")$`Pr(>Chi)`[2])
2 [1] 0.2547446
3 > print(anova(MLR_without_status, MLR_full, test="LRT")$`Pr(>Chi)`[2])
4 [1] 3.716688e-26
5 > print(anova(MLR, MLR_without_status, test="LRT")$`Pr(>Chi)`[2])
6 [1] 3.38473e-28

```

Listing 6: Conducting LRTs in R

From the result, we can derive these conclusions:

- The first comparison yields $Pr(> Chi) = 0.2547$, indicating that removing the two predictors **Screen** and **Storage.type** significantly improves our model.
- The second comparison yields $Pr(> Chi) < 0.01$, indicating that removing the predictors **Screen**, **Storage.type**, along with **Status** does not significantly improve our model.
- The third comparison yields $Pr(> Chi) < 0.01$, indicating that removing **Status** from the new model (already removed **Screen** and **Storage.type**) does not significantly improve it.

Our next step is to perform a multicollinearity test between independent variables using VIF (Variance Inflation Factor) test. When $GVIF$ of the i^{th} variable is equal to 1, it is not correlated to the remaining ones, meaning that multicollinearity does not exist for that variable. For categorical variables (especially non-binary ones), we will rely on

$$ScaleGVIF = GVIF^{1/2Df}$$

(Scale Generalized VIF, where Df is the degree of freedom of the variable) to make GVIFs comparable across dimensions. If a categorical variable has n classes, the Df value will be $n - 1$. The threshold for **Scaled GVIF** value is 2.5, meaning that **Scaled GVIF** value of a variable that is more than 2.5 is highly correlated and needs to be corrected.

```
1 vif_values <- vif(MLR)
2 print(vif_values)
```

Listing 7: VIF conducted in R

```
1 ----- Performing multicollinearity test with VIF -----
2               GVIF Df GVIF^(1/(2*Df))
3 Status      1.682301  1      1.297035
4 Model    2307.573534 52      1.077303
5 CPU       65.554868 13      1.174544
6 RAM       4.022136  1      2.005526
7 Storage   4.193817  1      2.047881
8 GPU      78.950998 13      1.182974
9 Touch     1.985503  1      1.409079
```

Listing 8: VIF output

In the summary, it is clear that except for the predictors **RAM** and **Storage** (which are our only two numerical variables), all other predictors have little to no correlations with other independent variables. However, from our domain knowledge of how people usually considering laptop price, RAM and Storage should not be excluded from the model. Therefore, no predictors are removed from the VIF test.

Now with the two predictors **Screen** and **Storage.type** removed, we can perform our MLR model again.

```
1 MLR_train_predictions <- predict(MLR, train_data)
2 MLR_test_predictions <- predict(MLR, test_data)
3 accuracy_MLR <- data.frame(Actual = test_data$Final.Price, Predicted = MLR_test_predictions)
4
5 MAPE_MLR <- mape(accuracy_MLR$Actual, accuracy_MLR$Predicted)
6 cat("MAPE:", MAPE_MLR, "\n")
7
8 print(summary(MLR))
9 cat("train_rmse = ", summary(MLR)$sigma, "\ntest_rmse = ", rmse(accuracy_MLR$Actual, accuracy_MLR$Predicted), "\n")
```

Listing 9: Retrain the model

```
1 ----- Multi-linear regression -----
2 MAPE: 0.198197
3 train_rmse = 300.7007
4 test_rmse = 306.0777
5
6
7 Residual standard error: 300.7 on 1133 degrees of freedom
8 Multiple R-squared: 0.8847, Adjusted R-squared: 0.8764
9 F-statistic: 106 on 82 and 1133 DF, p-value: < 2.2e-16
```

Listing 10: New summary of the model

As shown in the picture, the removal of the two predictors did not significantly affect our MLR model. The accuracy MAPE is slightly better (from 19.87% to 19.82%) on the testing set. The residual mean squared error (RMSE) on the training and testing set does not have significant difference also. However, for both model, the RMSE on the testing set is slightly larger than that on the training set, indicating there is slight overfit in the model.

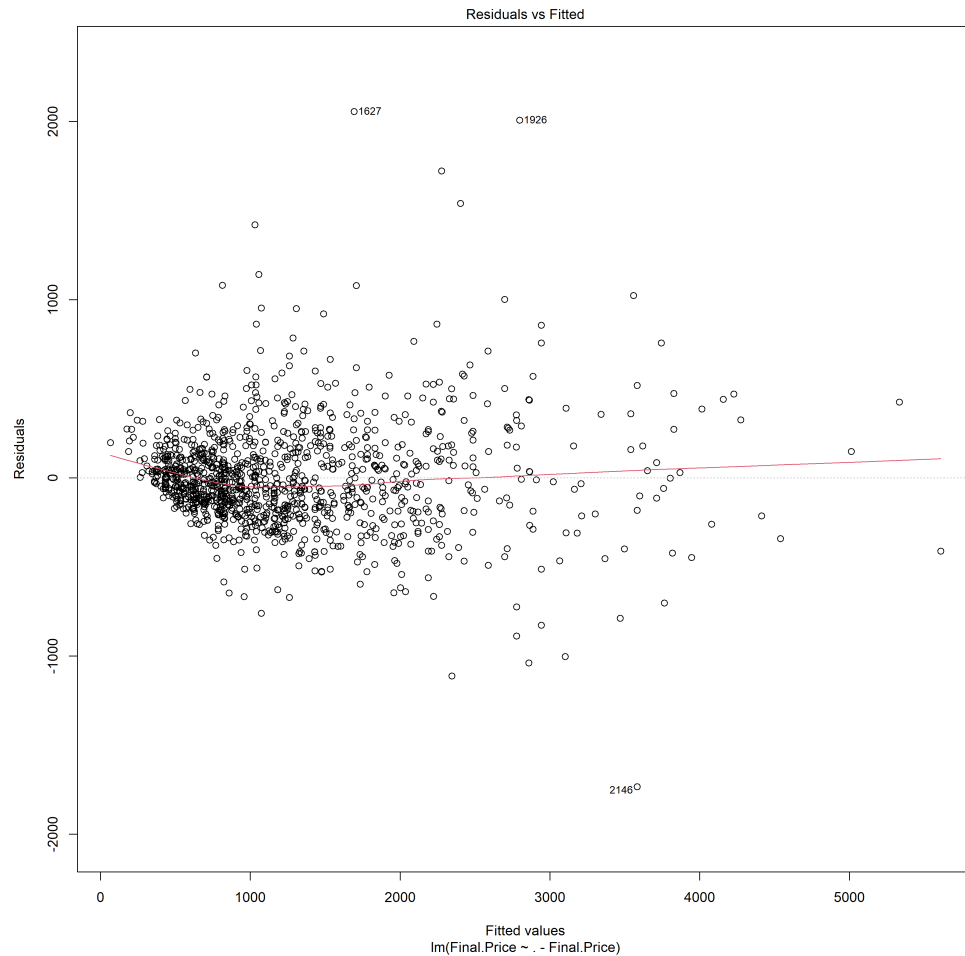


Figure 3: Residual vs. Fitted, Multi-linear Regression Model

From the plot, we can figure out:

- **Linearity:** The red line showed some curvedness when the price is low, indicating that the model is not fully linear and maybe violates the assumption of a reliable linear regression model.
- **Homoscedasticity:** the spread of the residuals increases as the fitted values increases, creating a funnel-like shape.
- **Outliers:** the price of 1627, 1926, and 2146 USD have extremely large residuals, indicating that these values may be outliers or statistically significant points.

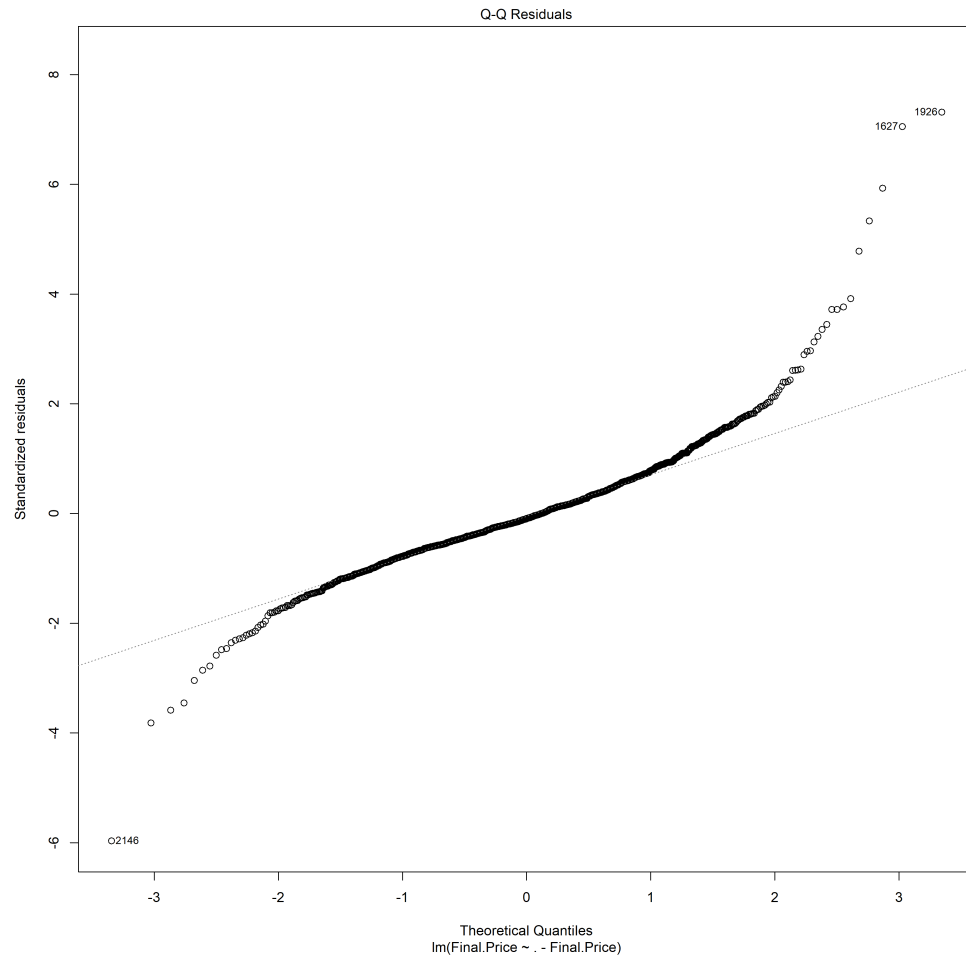


Figure 4: Q-Q Plot, Multi-linear Regression Model

The Q-Q plot is designated for testing normality of the residuals. From the plot:

- The x-axis represents the theoretical quantiles, which would be the quantiles from a standard normal distribution in this case.
- The y-axis represents the standardized residuals, which are the quantiles of the residuals of the model.
- The dashed line in the plot is the reference line where the points would lie if the residuals were perfectly normally distributed.

It seems that the residuals mostly follow the reference line, suggesting they are close to normally distributed. However, there are some large deviations, especially in the tails, which might suggest the residuals have heavier tails than a normal distribution. We can check the distribution of the residuals by another plots: the PDF of the residuals.

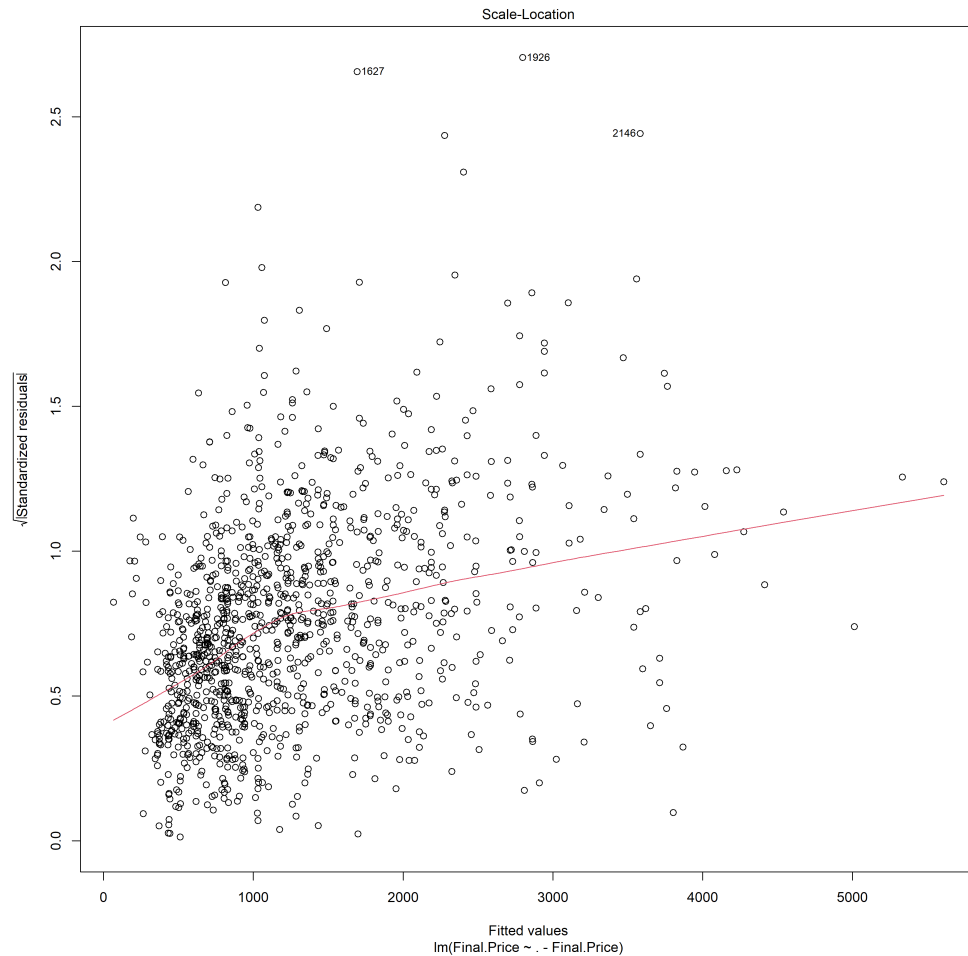


Figure 5: Scale-Location Plot, Multi-linear Regression Model

The Scale-Location Plot shows the funnel-shaped distribution of the points, which mean the residuals are not randomized, but rather spreads out as the fitted value increases. This violates homoscedasticity condition of the linear regression model. Moreover, the red line shows a noticeable slope at small fitted values (Final.Price), then it become horizontal afterwards, indicating that the residual mean is not approximately zero, especially in the case of predicting low prices. If the variance of the residuals is not constant, and the residual mean is not approximately zero, the model is fairly unreliable.

So far, there are two main outliers/influential data points appear in the residual plots: 1627 and 1926. There is proof that they distort the the true relationship between the response and the explanatory variables. Removing them from the data/model could reduce the overall residual variance, but introduce more bias to our model.

We will finally test the homoscedasticity and independence based on statistical tests:

```
1 # Homoscedasticity Test
2 print(ncvTest(MLR))
3
4 # Independence Test (Autocorrelation Test)
5 print(durbinWatsonTest(MLR))
```

Listing 11: Homoscedasticity and Independence Test

```

1 Non-constant Variance Score Test
2 Variance formula: ~ fitted.values
3 Chisquare = 294.7874, Df = 1, p = < 2.22e-16
4 lag Autocorrelation D-W Statistic p-value
5 1 0.1416164 1.715933 0
6 Alternative hypothesis: rho != 0

```

Listing 12: Homoscedasticity and Independence Test Result

The output is fairly expected, as the residuals can be proven heteroscedasticity from the plot. The Durbin-Watson Test tells us that there is some correlation on our dataset (which is further confirmed by the VIF test), but the D-W Statistics value is 1.72, which is acceptable in the range of 1.5 to 2.5.

From now on, we will make use of the remaining predictors.

5.2 Weighted multi-linear regression and log-transformation

As not all of the assumptions about the analysis of residuals are satisfied (specifically the homoscedasticity condition), it is time we think about another generalized linear model. Firstly we will perform log-transforming only the response variable (Final.Price), as log transformations can be used in regression models to deal with heteroscedasticity. It works by taking the logarithm of the dependent variable which compresses larger values, reducing the impact of extreme values, normalized the residuals' distribution, and making the variance of residuals approximately constant (hopefully). We will also use Weighted Least Squares (WLS) or weighted regression model instead of Ordinary Least Squares (OLS), allocating higher weight for small variances and smaller weight for the larger one. The reliability of applying weighted regression and log-transformation on the response variable is tested based on the Non-Constant Variance Test (ncvTest()) provided in R. It is based on the Breusch-Pagan famous test.

```

1 train_data_new <- train_data
2 train_data_new$Final.Price <- log(train_data_new$Final.Price) # log-transforming the response variable
3 MLR_new <- lm(Final.Price ~ ., data = train_data_new)
4
5 test_data_new <- test_data
6 test_data_new$Final.Price <- log(test_data_new$Final.Price)
7
8 wt <- 1 / lm(abs(MLR_new$residuals) ~ MLR_new$fitted.values)$fitted.values^2 # lower variance have bigger weight
9 wls_model <- lm(Final.Price ~ ., data = train_data_new, weights=wt)
10 print(summary(wls_model))

```

Listing 13: Train the weighted model

```

1
2 ----- Weighted Regression and Response Variable Transformation -----
3
4 Call:
5 lm(formula = Final.Price ~ ., data = train_data_new, weights = wt)
6
7 Weighted Residuals:
8      Min       1Q   Median       3Q      Max
9 -7.8432 -0.8198 -0.0566  0.7351  5.6881
10
11 Coefficients:
12              Estimate Std. Error t value Pr(>|t|)
13 (Intercept)    5.8955933   0.0929801   63.407 < 2e-16 ***
14 StatusRefurbished -0.2434185   0.0182816  -13.315 < 2e-16 ***
15 ModelAcer Chromebook  0.2344851   0.1129489    2.076 0.038116 *
16 ModelAcer ConceptD    0.5094973   0.1120459    4.547 6.02e-06 ***
17 ModelAcer Extensa    -0.2117154   0.0799155   -2.649 0.008180 **
18 ModelAcer Nitro       0.1021697   0.0847644    1.205 0.228325
19 ModelAcer Predator    0.1379343   0.0931198    1.481 0.138816
20 ModelAlurin Flex Advance -0.2217244   0.0917566   -2.416 0.015830 *
21 ModelApple MacBook Air  0.3236237   0.0746476    4.335 1.58e-05 ***
22 ModelApple MacBook Pro  0.5203456   0.0731681    7.112 2.03e-12 ***
23 ModelAsus Chromebook  0.2056552   0.0980639    2.097 0.036201 *
24 ModelAsus ExpertBook  0.2164418   0.0633118    3.419 0.000652 ***
25 ModelAsus F515        0.0557453   0.0800888    0.696 0.486544

```



26	ModelAsus ROG	0.2474925	0.0724361	3.417	0.000656	***
27	ModelAsus TUF	0.0949500	0.0728545	1.303	0.192743	
28	ModelAsus VivoBook	-0.0209397	0.0562790	-0.372	0.709911	
29	ModelAsus ZenBook	0.2835073	0.0668054	4.244	2.38e-05	***
30	ModelDell Latitude	0.4049030	0.0652336	6.207	7.56e-10	***
31	ModelDell Vostro	0.2133946	0.0685977	3.111	0.001912	**
32	ModelGigabyte Aero	0.2336511	0.0966073	2.419	0.015739	*
33	ModelGigabyte Aorus	0.1966590	0.0928664	2.118	0.034422	*
34	ModelHP 15S	-0.1018563	0.0555577	-1.833	0.067014	.
35	ModelHP 250	-0.0282330	0.0724706	-0.390	0.696921	
36	ModelHP 255	0.0868582	0.0893629	0.972	0.331272	
37	ModelHP Chromebook	0.0265352	0.1462171	0.181	0.856025	
38	ModelHP EliteBook	0.4746791	0.0648147	7.324	4.57e-13	***
39	ModelHP Omen	0.0648688	0.0807299	0.804	0.421838	
40	ModelHP Pavilion	0.2065643	0.0880786	2.345	0.019187	*
41	ModelHP ProBook	0.2722957	0.0716000	3.803	0.000151	***
42	ModelHP Victus	0.1143359	0.0804045	1.422	0.155299	
43	ModelHP Zbook	1.0655619	0.1052368	10.125	< 2e-16	***
44	ModelLenovo IdeaPad	0.0765500	0.0558600	1.370	0.170837	
45	ModelLenovo Legion	0.3505988	0.0825281	4.248	2.33e-05	***
46	ModelLenovo ThinkBook	0.4102849	0.0623966	6.575	7.39e-11	***
47	ModelLenovo ThinkPad	0.4948624	0.0564880	8.760	< 2e-16	***
48	ModelLenovo V15	-0.1127547	0.0616063	-1.830	0.067476	.
49	ModelLenovo Yoga	0.4219727	0.0798420	5.285	1.51e-07	***
50	ModelLG Gram	0.4080894	0.0832775	4.900	1.10e-06	***
51	ModelMedion Akoya	-0.1477253	0.0836805	-1.765	0.077774	.
52	ModelMicrosoft Surface Laptop	0.5270222	0.0753254	6.997	4.48e-12	***
53	ModelMicrosoft Surface Pro	0.5668559	0.0755056	7.507	1.22e-13	***
54	ModelMSI Creator	0.2248248	0.0809055	2.779	0.005545	**
55	ModelMSI Katana	0.1993735	0.0865806	2.303	0.021473	*
56	ModelMSI Leopard	0.0971009	0.0988398	0.982	0.326109	
57	ModelMSI Modern	0.0646566	0.0718126	0.900	0.368124	
58	ModelMSI Prestige	0.2856663	0.0724970	3.940	8.63e-05	***
59	ModelMSI Pulse	0.2324567	0.0897814	2.589	0.009745	**
60	ModelMSI Raider	0.3331188	0.0846121	3.937	8.75e-05	***
61	ModelMSI Stealth	0.2715644	0.0809177	3.356	0.000817	***
62	ModelMSI Summit	0.2696018	0.0847671	3.180	0.001510	**
63	ModelMSI Thin	-0.0123126	0.0970101	-0.127	0.899026	
64	ModelPcCom Revolt	0.1963520	0.0891471	2.203	0.027826	*
65	ModelRazer Blade	0.6417482	0.0778496	8.243	4.59e-16	***
66	ModelSamsung Galaxy Book	0.4919851	0.0867341	5.672	1.79e-08	***
67	CPUAMD Ryzen 3	0.1076263	0.0913458	1.178	0.238952	
68	CPUAMD Ryzen 5	0.3316209	0.0834943	3.972	7.58e-05	***
69	CPUAMD Ryzen 7	0.5078119	0.0843838	6.018	2.38e-09	***
70	CPUAMD Ryzen 9	0.6635161	0.0976956	6.792	1.79e-11	***
71	CPUApple M1	0.7598549	0.1222703	6.215	7.22e-10	***
72	CPUApple M2	0.9744969	0.1139215	8.554	< 2e-16	***
73	CPUIntel Celeron	-0.0577289	0.0957369	-0.603	0.546633	
74	CPUIntel Core i3	0.1850486	0.0845939	2.187	0.028910	*
75	CPUIntel Core i5	0.4137337	0.0810699	5.103	3.91e-07	***
76	CPUIntel Core i7	0.6012073	0.0826821	7.271	6.62e-13	***
77	CPUIntel Core i9	0.6925857	0.0941245	7.358	3.57e-13	***
78	CPUIntel Evo Core i5	0.4986956	0.0985621	5.060	4.90e-07	***
79	CPUIntel Evo Core i7	0.7632365	0.0914659	8.344	< 2e-16	***
80	RAM	0.0085646	0.0013518	6.336	3.40e-10	***
81	Storage	0.0003025	0.0000394	7.676	3.52e-14	***
82	GPUGTX 1650	0.0304361	0.0510689	0.596	0.551307	
83	GPUGTX 1660	0.0512087	0.1081168	0.474	0.635846	
84	GPURTX 2060	0.0572571	0.0843028	0.679	0.497160	
85	GPURTX 2070	0.3085997	0.0829132	3.722	0.000207	***
86	GPURTX 3050	0.3069351	0.0430058	7.137	1.70e-12	***
87	GPURTX 3060	0.3655056	0.0492682	7.419	2.31e-13	***
88	GPURTX 3070	0.5016260	0.0574028	8.739	< 2e-16	***
89	GPURTX 3080	0.6934676	0.0646552	10.726	< 2e-16	***
90	GPURTX 4050	0.3120849	0.0658701	4.738	2.43e-06	***
91	GPURTX 4060	0.3856409	0.0605646	6.367	2.79e-10	***
92	GPURTX 4070	0.4618239	0.0666406	6.930	7.04e-12	***
93	GPURTX 4080	0.7667933	0.0852830	8.991	< 2e-16	***

```

94 GPURTX 4090          0.7117006  0.0944187  7.538 9.76e-14 ***
95 TouchYes            0.1967651  0.0292057  6.737 2.56e-11 ***
96 ---
97 Signif. codes:  0    ***    0.001    **    0.01    *    0.05    .    0.1    1
98
99 Residual standard error: 1.387 on 1133 degrees of freedom
100 Multiple R-squared:  0.8757, Adjusted R-squared:  0.8667
101 F-statistic: 97.38 on 82 and 1133 DF,  p-value: < 2.2e-16

```

Listing 14: WLS Training output

Now we compare the plots of this model with the normal MLR model:

```

1 png('WeightedMLR%03d.png', width=12, height=12, units='in', res=300)
2 plot(wls_model, main = "Weighted Multi-linear Regression Model", ask = FALSE)
3 dev.off()

```

Listing 15: WLS residual plotting in R

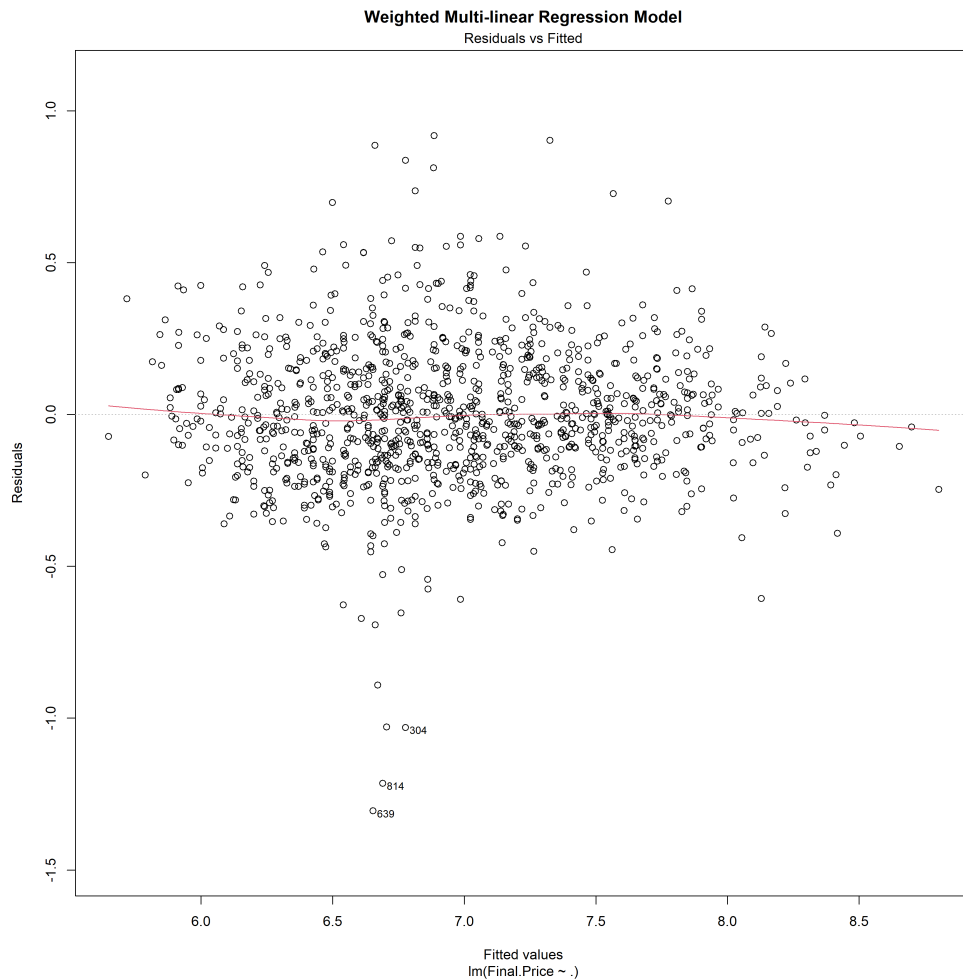


Figure 6: Residual vs. Fitted, Weighted MLR model

The residuals now appear to be more randomized and have constant variances around the dashed line. There are outliers that may need to be removed from the data/model since they mislead us towards spurious data. The red line

is also fairly straight and close to the dashed line, indicating linearity.

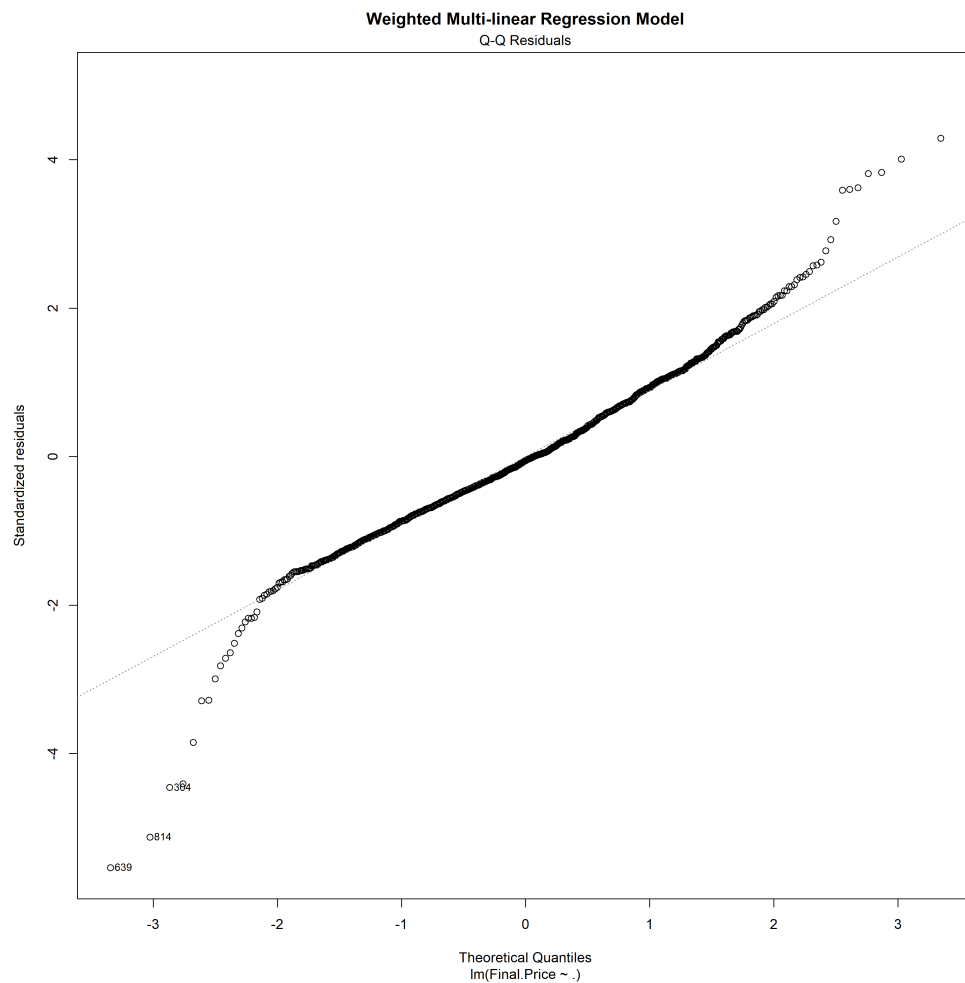


Figure 7: Q-Q Plot, Weighted MLR Model

The Q-Q plot resolves the problem of violating Normality constraint from the last model, as the residuals in this plot follow the reference line more closely than in the previous plot. Unfortunately, outliers still need to be handled for the distribution to become more normal.

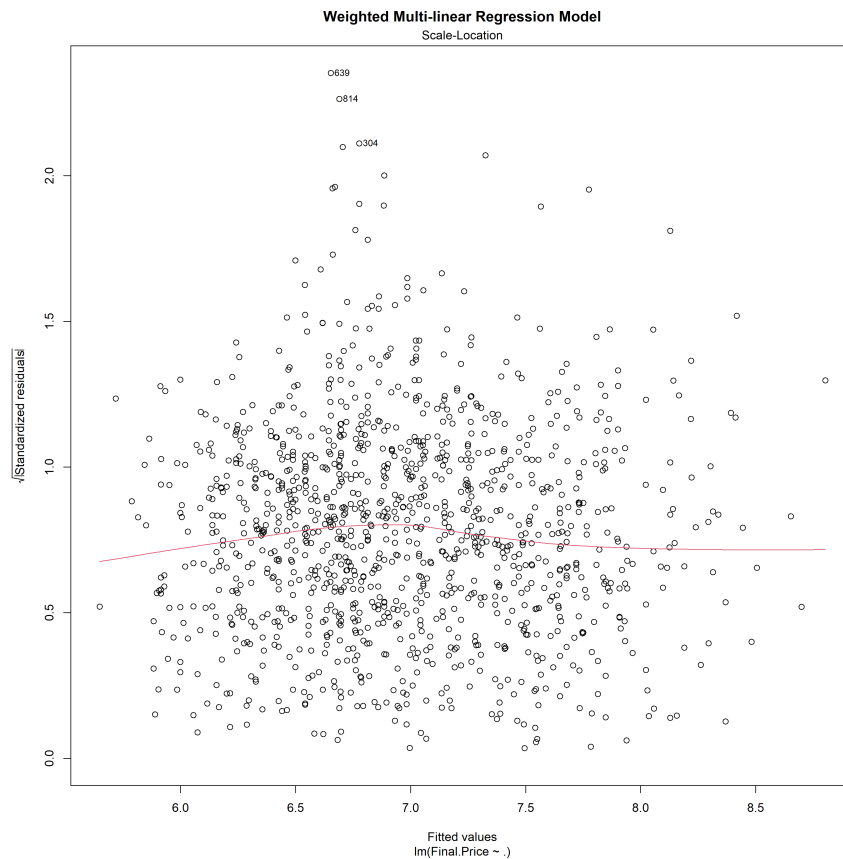


Figure 8: Scale-Location Plot, Weighted MLR Model

The square root of the standardized residuals seem to scatter randomly, and the red line stays fairly horizontal, indicating the residual mean stays approximately zero and acceptable constant residual variances.

Calculate the RMSE on the training and testing dataset:

```
1 train_predictions <- predict(wls_model, newdata = train_data_new)
2 new_MLR_predictions <- predict(wls_model, newdata = test_data_new)
3
4 cat("train_rmse = ", rmse(exp(train_predictions), exp(train_data_new$Final.Price)),
5 "\ntest_rmse = ", rmse(exp(new_MLR_predictions), exp(test_data_new$Final.Price)), "\n")
```

Listing 16: WLS Predictions on Testing dataset

```
1 train_rmse = 308.2831
2 test_rmse = 319.2129
```

Listing 17: WLS MSE Result

```
1 Non-constant Variance Score Test
2 Variance formula: ~ fitted.values
3 Chisquare = 1.308072, Df = 1, p = 0.25274
4
5 Durbin-Watson Test
6 lag Autocorrelation D-W Statistic p-value
7 1 0.1416164 1.715933 0
8 Alternative hypothesis: rho != 0
```

Listing 18: WLS Homoscedasticity and Independence Test

The p-value yields 0.253, reflecting that the weighted MLR model is statistically important and a suitable upgrade of MLR to prevent heteroscedasticity. In conclusion, we have somewhat solved the non-constant variance problem by generalizing the model, while figuring out the cause of heteroscedasticity is from outliers.

5.3 Lasso regression

Perform cross-validation for Lasso model to find the best lambda (the one minimizing the mean cross-validation error), along with predictions on both training and testing set:

```
1 lasso_model <- cv.glmnet(X_train_transformed, y_train, alpha = 1) # Set alpha to 1 for L1 regularization (Lasso)
2 lambda_value_L1 <- lasso_model$lambda.min # Cross-validation will find the suitable lambda
3 # that minimize the CV error for us. The default number of folds is
4 cat("Best lambda:", lambda_value_L1, "\n")
```

Listing 19: Cross-validation for Lasso Regression

```
1 ----- Lasso Regression -----
2 Best lambda: 0.7750494
3
4 Call: cv.glmnet(x = X_train_transformed, y = y_train, alpha = 1)
5
6 Measure: Mean-Squared Error
7
8      Lambda Index Measure   SE Nonzero
9 min  0.775    73  100337 7705      79
10 1se  9.555    46  107314 8725      68
11 MAPE: 0.1981423
12
13 Coefficient of Determination: 0.8844941
14 train_rmse = 316.7601
15 test_rmse = 306.4238
```

Listing 20: Result

While the accuracy of the prediction slightly increases, the RMSE on training set is slightly larger than that on testing set, meaning that the problem of overfitting is resolved. This is the trade-off between fitting and accuracy.

Cross-validation plot:

```
1 png('lasso_plot.png', width=12, height=12, units='in', res=300)
2 plot(lasso_model, main = "Lasso Model Plot", ask = FALSE)
3 dev.off()
```

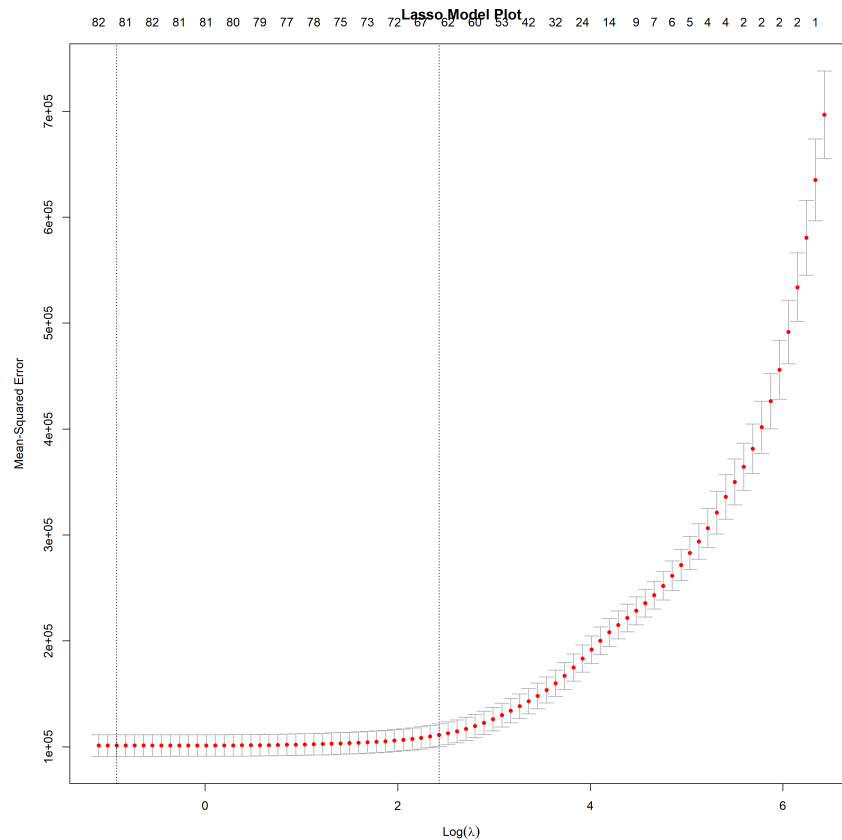


Figure 9: Cross-validation on Lasso model

The number above the plot shown how many predictors are left in the model. When λ is very high, all the coefficients are shrunk exactly to zero. The two vertical dashed lines are the $\log \lambda$ values corresponding to the λ_{min} (left dashed line) and λ_{1se} (right dashed line). λ_{1se} is the largest value of λ such that the error is within one standard error of the minimum. While λ_{min} gives the model with the smallest cross-validated error, it might also lead to a more complex model with a larger number of predictors (looking at the summary, there are 79 out of 84 nonzero coefficients). On the other hand, λ_{1se} gives a simpler model (68 out of 84 nonzero coefficients), which might be more interpretable and less likely to overfit the data. In this project, we work with λ_{min} . The grey whiskers for each red data point is the standard error.

5.4 Ridge regression

The similar code from Lasso regression is assigned to conduct the Ridge regression, only replacing *alpha* with 1 instead of 0.

```

1
2
3 ----- Ridge Regression -----
4 Best lambda: 62.86664
5
6 Call: cv.glmnet(x = X_train_transformed, y = y_train, alpha = 0)
7
8 Measure: Mean-Squared Error
9
10      Lambda Index Measure  SE Nonzero
11 min  62.87   100  100813 6468      84
12 1se  231.25    86  107102 6255      84

```

```

13 MAPE: 0.2002953
14
15 Coefficient of Determination: 0.8811915
16 train_rmse = 317.5106
17 test_rmse = 305.6023

```

Listing 21: Result

The ridge regression is usually more extreme in reducing overfitting, so it is expected for MAPE and the difference of `train_rmse` and `test_rmse` to increase.

Cross-validation plot:

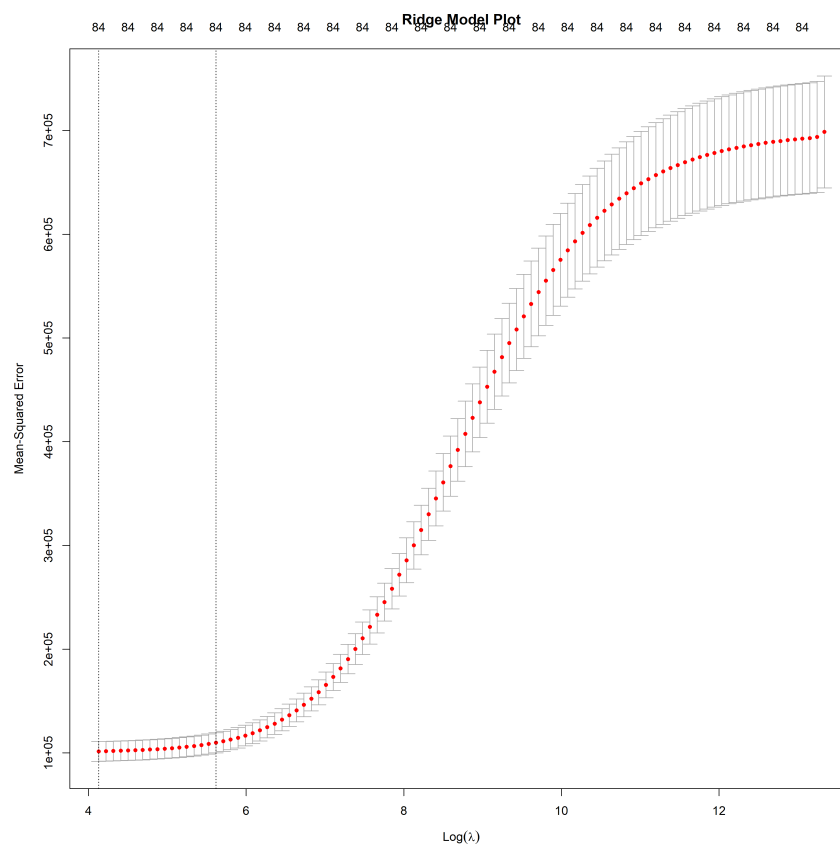


Figure 10: Cross-validation on Ridge model

As λ increases, RMSE increases, and the coefficient of each predictor decreases (but never reaches 0).

5.5 Elastic net

Note that the best controlling value α and λ for each form of regularization is estimated through cross-validation technique in our snippet.

```

1 alpha_seq <- seq(0, 1, by = 0.05) # For example, from 0 to 1 in steps of 0.05
2 elastic_models_list <- list()
3 for(i in seq_along(alpha_seq)) {
4   alpha <- alpha_seq[i]
5   elastic_cv_model <- cv.glmnet(x = X_train_transformed, y = y_train, alpha = alpha)
6   elastic_models_list[[paste0("alpha_", alpha)]] <- elastic_cv_model

```

```

7 }
8
9 # Initialize minimum RMSE of the cross-validation and best alpha corresponding
10 min_rmse <- Inf
11 best_alpha <- NA
12
13 # Loop over the results
14 for(alpha_name in names(elastic_models_list)) {
15   # Extract the alpha value from the name
16   alpha <- as.numeric(gsub("alpha_", "", alpha_name))
17   rmse <- sqrt(elastic_models_list[[alpha_name]]$cvm[elastic_models_list[[alpha_name]]$lambda == elastic_models_1
18   if(rmse < min_rmse) {
19     min_rmse <- rmse
20     best_alpha <- alpha
21   }
22 }
23
24 # Print the best alpha
25 cat("Best alpha:", best_alpha)

```

Listing 22: α Tuning for Elastic Net

```

1 elastic_model <- cv.glmnet(X_train_transformed, y_train, alpha = best_alpha)
2 best_lambda <- elastic_model$lambda.min

```

Listing 23: Elastic Net Cross-validation Training

```

1 ----- Elastic Net -----
2 Best alpha: 0.1
3 Call: cv.glmnet(x = X_train_transformed, y = y_train, alpha = best_alpha)
4
5 Measure: Mean-Squared Error
6
7
8      Lambda Index Measure   SE Nonzero
9 min    4.44    79  100010  7767      82
10 1se   79.33    48  106749  7997      73
11
12 MAPE: 0.198309
13 Coefficient of Determination: 0.8844744
14 train_rmse = 316.2432
15 test_rmse = 306.3241

```

Listing 24: Result

6 Discussion and extension

6.1 Discussion of the Multivariate Linear Regression model

The multiple linear regression model demonstrates encouraging capabilities in predicting final product prices. This initial evaluation offers valuable insights that can guide further refinement of the model, potentially leading to even better prediction accuracy in the future.

6.2 Extension and Comparison

In this section, we aim to test various models to assess their accuracy and compare with the performance of Multiple Linear Regression (section 5). By experimenting with different models, we can gain insights into which algorithms are best suited for our dataset and task at hand.

Models	Accuracy
Decision tree	66.25769 %
Gradient Boosting	71.31021 %
Support Vector Machine	73.06723%

6.3 Decision tree

Because Decision tree can contain categorical data and numeric data. From the dataset, we select 7 columns "CPU", "RAM", "Storage.type", "Screen", "Status", "Touch", "Storage" to predict the final price.

```
1 # Set the seed for reproducibility
2 set.seed(123)
3
4 # Split the data into training and testing sets
5 train_indices <- createDataPartition(data$Final.Price,
6                                     p = 0.9, list = FALSE)
7 train_data <- data[train_indices, ]
8 test_data <- data[-train_indices, ]
9
10 # Select the subset of features
11 selected_features <- c("CPU", "RAM", "Storage.type", "Screen",
12                       "Status", "Touch", "Storage")
```

Using `createDataPartition()` function from the caret package in R to partition a data frame into training and testing sets for model building. We use `p = 0.9` specifies the percentage of data to be used for training and `FALSE` to store the result as a vector of indices.

```
1 # Create a decision tree regression model
2 tree_model <- rpart(Final.Price ~ .,
3                     data = preprocessed_train_data)
```

Separating train data and test data from selected features above, we use `rpart()` function in rpart packages to create a decision tree model, which predicts the final prices based on the selected variables.

```
1 # Make predictions on the testing data
2 predictions <- predict(tree_model,
3                       newdata = preprocessed_test_data)
```

Using the `predict()` function to make predictions on the train data using the previously built decision tree model. After that, using the **Calculate the Mean Absolute Percentage Error (MAPE)** to measure the accuracy of the predicted data, this formula calculates the average of the absolute percentage error for all predictions.

```
1 MAPE <- mean(abs((accuracy$Actual - accuracy$Predicted)
2               / accuracy$Actual)) * 100
```

The Mean Absolute Percentage Error (MAPE): 33.74231 %

```
> head(accuracy,10)
      Actual Predicted
2         299  464.6232
6        1699 2327.5110
17         799 1223.3494
28         999 1223.3494
31         789 1223.3494
35        2249 2327.5110
43        1089 1223.3494
46         549  775.8160
51        1219  775.8160
57         399  464.6232
```

Figure 10: The actual and predicted data

6.4 Gradient Boosting

Similarly to Decision tree, Gradient Boosting Machines use 7 columns include categorical variables and numeric variables to predict the final price: "CPU", "RAM", "Storage.type", "Screen", "Status", "Touch", "Storage".

```
1 # Create a gradient boosting regression model
2 gbm_model <- train(Final.Price ~.,
3   data = preprocessed_train_data, method = "gbm", trControl =
4   trainControl(method = "cv", number = 5), verbose = FALSE)
```

After separating train data and test data with $p = 0.9$, we use **train()** function in caret packages to train model with method = "gbm", indicating Gradient Boosting Machine algorithm.

Using **predict()** function to predict the data based on model "gbm model" and the MAPE formula to calculate the absolute error between the test data and predicted data.

The Mean Absolute Percentage Error (MAPE): 28.68979 %.

```
> cat("Mean Absolute Percentage Error (MAPE):", MAPE, "%\n")
Mean Absolute Percentage Error (MAPE): 28.68979 %
```

```
> head(accuracy,10)
      Actual Predicted
1         299  477.3729
2        1699 2595.8939
3         799 1060.3867
4         999 1060.3867
5         789 1060.3867
6        2249 2595.8939
7        1089 1449.3642
8         549  603.8437
9        1219  857.6893
10        399  477.3729
```

Figure 11: The result of Gradient Boosting Algorithm

6.5 Support Vector Machine

In this model, we use 7 columns to analyze the data, these columns are: "CPU", "RAM", "Storage.type", "Screen", "Status", "Touch", "Storage". Then, we split the figure into two sets: training set and testing set with $p = 0.9$ which means the size of the training set is equal to 90% of the initial set.

```
1 # Split the data into training and testing sets
2 train_indices <- createDataPartition(data$Final.Price, p = 0.9,
3 list = FALSE)
4 train_data <- data[train_indices, ]
5 test_data <- data[-train_indices, ]
6
7 # Print the dimensions of the training and testing sets
8 cat("Training data dimensions:", dim(train_data), "\n")
9 cat("Testing data dimensions:", dim(test_data), "\n")
10
11 # Select the subset of features
12 selected_features <- c("CPU", "RAM",
13 "Storage.type", "Screen", "Status",
14 "Touch", "Storage")
```

We preprocessed the test data by delete its Final.price column and then create a Support Vector Machine (SVM) regression model.

```
1 # Preprocess the data
2 preprocessed_train_data <- train_data[, c("Final.Price",
3 selected_features)]
4 preprocessed_test_data <- test_data[, selected_features]
5
6 # Create an SVM regression model
7 svm_model <- svm(Final.Price ~ ., data = preprocessed_train_data,
8 kernel = "linear")
```

Finally, we use the `predict()` function to make predictions on the train data using the previously SVM model. After that, we calculate the **Mean Absolute Percentage Error (MAPE)** to measure the accuracy of the predicted data.

```
1 # Make predictions on the testing data
2 predictions <- predict(svm_model,
3 newdata = preprocessed_test_data)
4
```



```
5 # Calculate the accuracy metrics
6 accuracy <- data.frame(Actual = test_data$Final.Price,
7                         Predicted = predictions)
8
9 MAPE <- mean(abs((accuracy$Actual - accuracy$Predicted)
10              / accuracy$Actual)) * 100
```

The Mean Absolute Percentage Error (MAPE): 26.93277

```
> head(accuracy,10)
      Actual Predicted
2       299  413.9707
6      1699 2131.4206
17       799 1074.9226
28       999 1074.9226
31       789 1074.9226
35      2249 2131.4206
43      1089 1319.5089
46       549  798.2699
51      1219 1155.4167
57       399  413.9707
```

Figure 12: The result of Support Vector Machine

7 Data and Code availability

Data: [Laptops Price Dataset](#)

References

- [1] [R Tutorial](#)
- [2] [Decision Tree](#)
- [3] [Correlation Matrix](#)
- [4] [Likelihood Ratio Test](#)
- [5] [Interpretation of CV plot for Lasso Regression](#)
- [6] [Residuals and Diagnosis for Linear Regression](#)
- [7] [Distinction Between Outliers and High Leverage Observations](#)
- [8] [Collinearity, VIF and GVIF](#)